Treinamento Front-End – TypeScript - Angular

# Sumário

O framework Angular ..... 3
Angular: Um Framework para Desenvolvimento de Aplicações Web Modernas ..... 3
Mudanças na estruturas dos projetos ..... 4
A arquitetura do framework Angular ..... 6
Configuração do ambiente ..... 8
Configuração do Projeto ..... 10
Data Binding ..... 16
Interpolação (Interpolation) ..... 17
Routing ..... 19
Ligação/Vinculação de dados baseada em propriedades (Property Binding) ..... 21
Vinculação/ligação por evento (Event Binding) ..... 23
Vinculação/ligação de dados bidirecional (Two-Way Data Binding) ..... 26
Diretivas ..... 29
Diretivas de estrutura ..... 29
*ngIf ..... 29
ngIf else ..... 30
ngIf then else ..... 31
ngFor ..... 33
Diretivas de atributo ..... 40
ngStyle ..... 44
Pipes ..... 47
Services ..... 51
O que é um service ..... 51
Para que services são utilizados ..... 52
Vantagens ao se usar services ..... 52
Como criar um service no Angular ..... 52
Angular Dependency Injection ..... 57
O que é dependência ..... 58
Angular Dependency Injection -definição ..... 58
Estrutura da Angular Dependency Injection ..... 61
Como funciona a injeção de dependência no Angular ..... 62
Como usar a dependency Injection (injeção de dependência) ..... 62
Injetando Service em Service ..... 64

Referencias: ..... 70

## O framework Angular

### Angular: Um Framework para Desenvolvimento de Aplicações Web Modernas

**Angular** é um framework open-source para o desenvolvimento de aplicações web, criado e mantido pelo Google. Ele foi projetado para facilitar a criação de aplicações web dinâmicas, de alta performance e escaláveis. Desde a sua versão 2, Angular passou a ser reescrito em **TypeScript**, oferecendo uma base sólida e moderna para o desenvolvimento de software. A versão mais recente do framework, Angular 19.*, apresenta aprimoramentos contínuos de desempenho e usabilidade.

A arquitetura do Angular é baseada em componentes, que são unidades reutilizáveis de código que encapsulam uma parte da interface do usuário e sua lógica associada. Cada componente do Angular contém um template HTML, uma classe TypeScript e uma folha de estilo CSS ou SCSS, o que facilita a manutenção e a escalabilidade das aplicações. Essa abordagem baseada em componentes é um dos principais fatores que torna o Angular adequado para o desenvolvimento de grandes aplicações de maneira organizada e eficiente.

O framework adota o modelo **Single Page Application (SPA)**, no qual toda a navegação entre diferentes telas é feita sem recarregar a página inteira, proporcionando uma experiência de usuário mais fluida e interativa. A navegação entre as páginas é gerida pelo sistema de **roteamento** do Angular, que permite carregar dinamicamente componentes com base nas mudanças de URL.

A **injeção de dependência** (DI) é um dos conceitos centrais do Angular, permitindo a separação das responsabilidades entre os diferentes módulos e a reutilização eficiente de serviços e outros objetos. Com DI, os componentes e serviços do Angular podem ser facilmente configurados e testados, promovendo um ambiente de desenvolvimento ágil e modular.

Uma característica fundamental do Angular é o seu uso de **RxJS**, uma biblioteca para programação reativa, que facilita a manipulação de fluxos de

dados assíncronos e eventos, como requisições HTTP e interações do usuário. O Angular integra o RxJS para lidar com observáveis, que são usados em grande parte das operações assíncronas do framework, proporcionando maior controle e eficiência no fluxo de dados.

Além disso, o **HttpClient** do Angular permite que as aplicações se comuniquem com servidores externos por meio de APIs RESTful de maneira simples e segura. A comunicação com servidores pode ser feita por meio de chamadas HTTP assíncronas, utilizando métodos como **GET**, **POST**, **PUT** e **DELETE**, facilitando a integração com serviços de backend.

O Angular também suporta **Testes Unitários** e **Testes de Integração**, com ferramentas como o **Karma** e o **Jasmine**, que ajudam a garantir a qualidade e a estabilidade das aplicações. A ferramenta **Angular CLI** (Command Line Interface) oferece comandos para gerar componentes, serviços e outros elementos do projeto, além de facilitar o processo de build e deployment.

A documentação oficial do Angular enfatiza o uso de boas práticas, como a modularização de código e a reutilização de componentes e serviços, para garantir a escalabilidade e a manutenção eficiente de projetos. Em sua versão mais recente, o Angular também introduziu melhorias no desempenho de build, na compactação de pacotes e na inicialização de aplicações, com o objetivo de reduzir o tempo de carregamento e melhorar a experiência do usuário.

### Mudanças na estruturas dos projetos

O **Angular 17** é a versão mais recente do framework de desenvolvimento front-end mantido pelo Google, lançada em 8 de novembro de 2023. Esta atualização introduziu diversas melhorias significativas, focando principalmente em desempenho, experiência do desenvolvedor e recursos avançados para aplicações web modernas.

**Principais Novidades do Angular 17:**

1. **Componentes Standalone como Padrão:**

- A partir do Angular 17, componentes, diretivas e pipes são standalone por padrão, eliminando a necessidade de módulos Angular (NgModules) para a maioria dos casos de uso. Essa mudança simplifica a estrutura do aplicativo e reduz a complexidade do código.

2. **Nova Sintaxe para Controle de Fluxo em Templates:**
    - O Angular 17 apresenta uma nova sintaxe para controle de fluxo em templates, substituindo as tradicionais diretivas estruturais como *ngIf e *ngFor. Essa mudança visa simplificar a leitura e manutenção do código, além de permitir uma detecção de mudanças mais eficiente.
3. **Lazy Load de Partes da Página:**
    - A introdução do carregamento preguiçoso de partes da página permite que apenas os elementos necessários sejam carregados inicialmente, reduzindo o tempo de carregamento da página e melhorando a performance geral da aplicação.
4. **Suporte Aprimorado para Renderização no Lado do Servidor (SSR):**
    - O Angular 17 oferece uma experiência SSR renovada com melhor desempenho e flexibilidade. A nova API de visualizações deferíveis permite renderizar partes da aplicação no lado do cliente, otimizando o tempo de carregamento inicial e a experiência do usuário.
5. **Compilação Mais Rápida:**
    - A velocidade de compilação foi significativamente aprimorada, reduzindo o tempo de espera durante o desenvolvimento e aumentando a produtividade dos desenvolvedores.
6. **Adoção do esbuild no Angular CLI:**

- O Angular CLI agora utiliza o esbuild, uma ferramenta de bundling e minificação de código, resultando em builds mais rápidos e eficientes.

7. **Aprimoramentos na Experiência do Desenvolvedor:**
    - O Angular 17 introduziu uma sintaxe de template mais intuitiva e concisa, facilitando a criação e o gerenciamento de templates. Além disso, as APIs autônomas permitem o uso de diretivas, componentes e serviços sem a necessidade de módulos, simplificando a organização e a reutilização de código.

Essas melhorias posicionam o Angular 17 como uma ferramenta ainda mais poderosa e eficiente para o desenvolvimento de aplicações web modernas, atendendo às crescentes demandas por desempenho, escalabilidade e uma experiência de desenvolvimento aprimorada.

## A arquitetura do framework Angular

A arquitetura do framework Angular é baseada em componentes, módulos e serviços, seguindo um padrão de design modular e orientado a componentes.

### 1. Componentes

Os componentes são os blocos fundamentais do Angular. Eles encapsulam a lógica, o template (HTML) e os estilos (CSS) de uma parte da interface do usuário. Cada componente é uma classe TypeScript decorada com @Component, que define metadados como o seletor, o template e os estilos.

Estudo: De acordo com Freeman (2020), a arquitetura baseada em componentes promove a reutilização de código e a separação de preocupações, facilitando a manutenção e a escalabilidade de aplicações web complexas.

### 2. Módulos

Os módulos (@NgModule) são usados para organizar a aplicação em blocos funcionais. Um módulo declara componentes, diretivas, pipes e serviços, e pode importar outros módulos. O AppModule é o módulo raiz que inicializa a aplicação. Obs. Esta modalidade vigorou ate a versão 16 -ainda existem aplicações nesta modalidade que proporcionam uma migração gradual’

Estudo: Segundo Misko Hevery, um dos criadores do Angular, a modularidade é essencial para aplicações grandes, pois permite o carregamento sob demanda (lazy loading) e a divisão de responsabilidades (Hevery & Abrons, 2016).

### 3. Serviços e Injeção de Dependência

Serviços são classes que encapsulam lógica de negócios, como chamadas HTTP, manipulação de dados ou autenticação. Eles são injetados em componentes ou outros serviços por meio do sistema de injeção de dependência (DI) do Angular.

Estudo: Fowler (2004) destaca que a injeção de dependência promove o desacoplamento e a testabilidade, princípios que o Angular adota fortemente em sua arquitetura.

**4. Roteamento**

O Angular Router permite a navegação entre diferentes componentes com base na URL. Ele suporta rotas parametrizadas, lazy loading de módulos e guardas de rota para controle de acesso.

Estudo: De acordo com um estudo de caso da Google (2018), o roteamento avançado do Angular é crucial para aplicações de página única (SPAs), proporcionando uma experiência de usuário fluida e eficiente.

**5. Two-Way Data Binding**

O Angular utiliza two-way data binding, sincronizando automaticamente o modelo (model) e a visão (view). Isso é feito por meio da diretiva ngModel, que simplifica a manipulação de formulários e a interação com o usuário.

Estudo: Um artigo de Smith et al. (2017) compara o two-way data binding do Angular com outras abordagens, concluindo que ele reduz a quantidade de código boilerplate e melhora a produtividade do desenvolvedor.

**6. Diretivas**

Diretivas são instruções no DOM que estendem o comportamento de elementos HTML. O Angular oferece diretivas embutidas, como ngIf e ngFor, e permite a criação de diretivas personalizadas.

Estudo: De acordo com um estudo de Gupta (2019), o uso de diretivas personalizadas pode aumentar a modularidade e a reutilização de código em aplicações Angular.

**7. Pipes**

Pipes são usados para transformar dados diretamente nos templates, como formatação de datas, moedas e textos. O Angular inclui pipes embutidos e permite a criação de pipes personalizados.

Estudo: Um artigo de Kumar (2020) destaca que pipes são uma ferramenta poderosa para manter a lógica de formatação fora dos componentes, seguindo o princípio de responsabilidade única.

**8. Change Detection**

O Angular utiliza um mecanismo de detecção de mudanças eficiente, que atualiza a interface do usuário sempre que o estado da aplicação muda. Isso é feito por meio de zone.js, que monitora operações assíncronas.

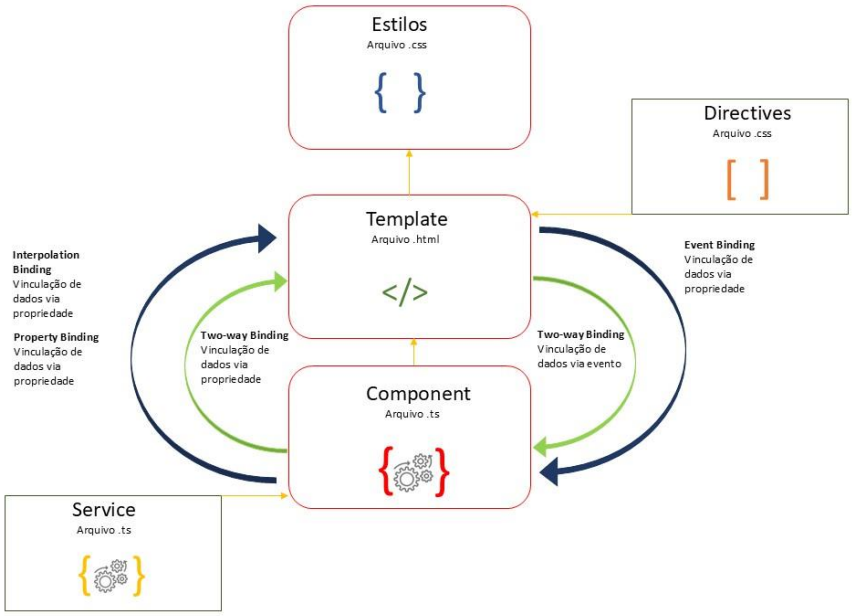
Estudo: Um estudo de caso da Microsoft (2019) comparou o desempenho do Angular com outros frameworks e destacou que a detecção de mudanças baseada em zones é uma das razões para sua eficiência em aplicações complexas.

**9. RxJS e Programação Reativa**

O Angular integra a biblioteca RxJS para lidar com operações assíncronas e programação reativa. Observables e operadores RxJS são amplamente usados em serviços e componentes.

Estudo: De acordo com um artigo de Meijer (2016), a programação reativa com RxJS simplifica o gerenciamento de fluxos de dados assíncronos, tornando o código mais legível e manutenível.

A arquitetura do Angular é projetada para ser modular, escalável e de fácil manutenção, seguindo boas práticas de desenvolvimento de software. Estudos e casos de uso demonstram que sua abordagem baseada em componentes, combinada com ferramentas como injeção de dependência, roteamento e RxJS, torna-o uma escolha robusta para aplicações web modernas. Observe o diagrama abaixo:




## Configuração do ambiente

Para fazer uso do Angular, é necessário ter instalados em seu computador:

- Nodejs
- Npm
- Angular CLI
- IDE para escrever seu código

**Nodejs**

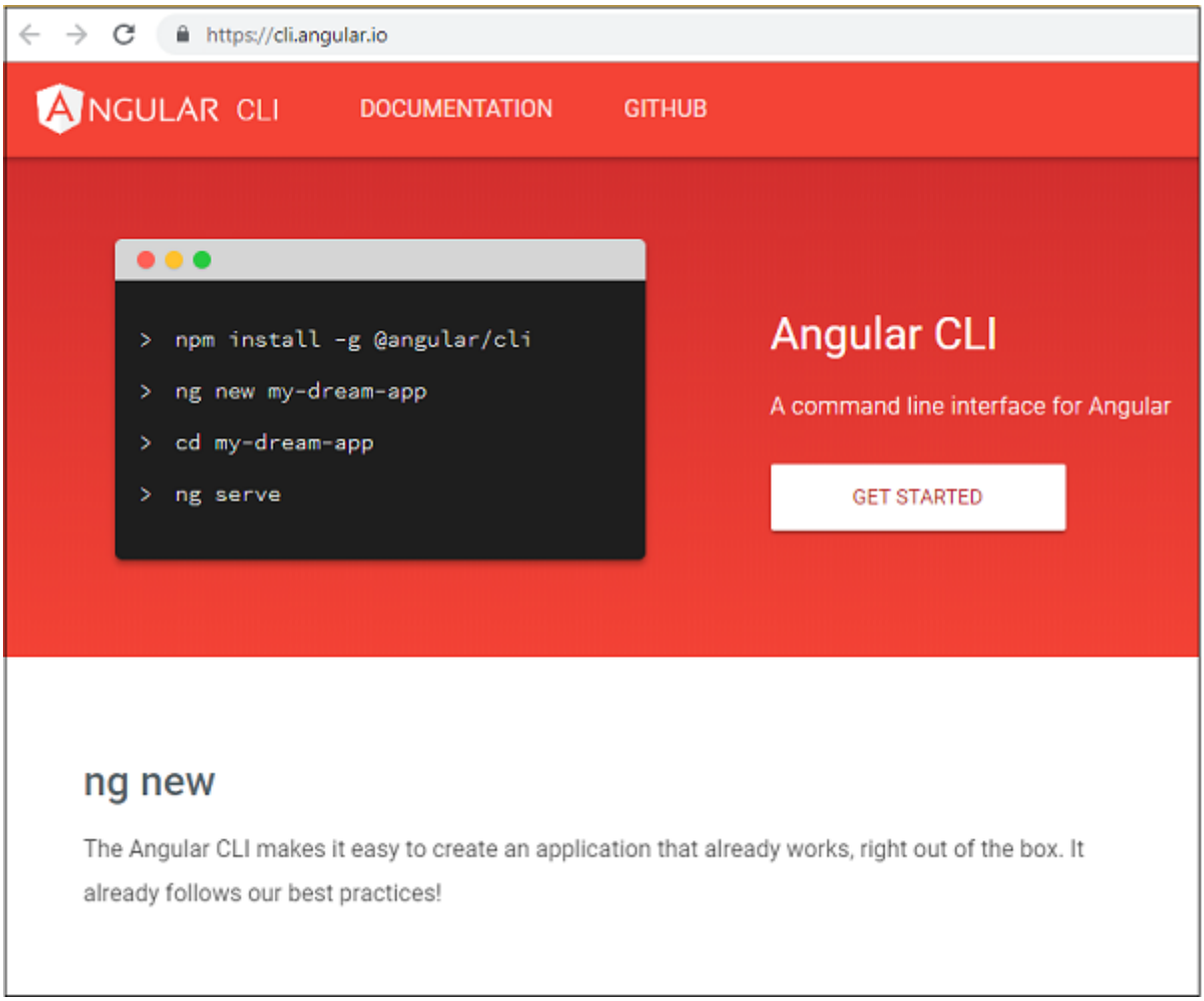
Para verificar se o nodejs está instalado no seu sistema, digite **node -v** no terminal. Isso ajudará você a ver a versão do nodejs atualmente instalada no seu sistema.

```
C:\>node -v
```

Com base no seu sistema operacional, instale o pacote necessário. Depois que o nodejs estiver instalado, o npm também será instalado junto com ele. Para verificar se o npm está instalado ou não, digite npm –v no terminal, conforme indicado abaixo. Ele exibirá a versão do npm.

```
C:\>npm -v
```

As instalações do Angular são muito simples com a ajuda da CLI angular. Visite a página inicial https://cli.angular.io/ do angular para obter a referência do comando.

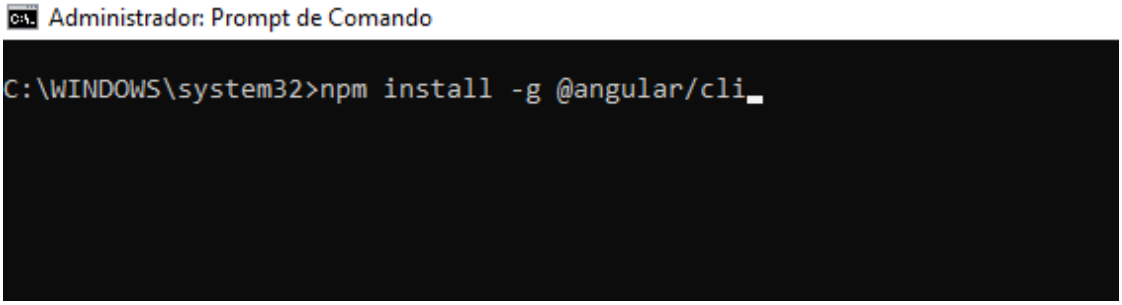


Digite ***npm install –g @angular/cli*** no prompt de comando para instalar o angular cli no seu sistema. Vai demorar um pouco para instalar e, uma vez feito, você pode verificar a versão usando o comando abaixo:

```
npm install -g @angular/cli //comando instalar o Angular
ng new nome_do_projeto // nome do projeto
cd nome-da-pasta
ng serve
```

Os comandos acima ajudam a obter a configuração do projeto no Angular.

Crie uma pasta chamada **projetos-angular** e instalar ***angular/cli*** como mostrado abaixo




Depois que a instalação estiver concluída, verifique os detalhes dos pacotes instalados usando o comando ***ng version***. O comando a versão para o Angular CLI e as versões dos pacotes disponíveis para o Angular.

**Atualização angular**

Caso você esteja usando uma versão do Angular que seja antiga e queira atualizar para o Angular na sua versão atual, abaixo está o comando que atualizará seu aplicativo para a versão recente do Angular:

```
ng update @angular/cli
```

## Configuração do Projeto

**Projeto angular-alpha**

Crie uma pasta em seu local de preferencia. Essa pasta abrigará os projetos angular. Para criar uma pasta na área de trabalho – via prompt de comando – basta inserir no prompt as seguintes intruções:

```
C:\Users\SeuUsuario\Desktop\mkdir projetos-angular
C:\Users\SeuUsuario\Desktop\cd projetos-angular
C:\Users\SeuUsuario\Desktop\projetos-angular\ng new angular-alpha
```

O conjunto de instruções acima mostra que: devemos encontrar e inserir no prompt o path (caminho) completo do diretório “Area de trabalho (Desktop)”. Na sequencia, inserimos o comando mkdir e o nome da nossa

pasta (projetos-angular). Feito isso, basta clicar Enter em seu teclado. Sua pasta para os projetos Angular está criada.

Na sequencia, basta inserir ainda no Prompt de comando a instrução: *cd projetos-angular.* Para criar um projeto Angular, foi usado o seguinte comando:  a partir do prompt de comando:

```
ng new angular-alpha
```

Use o *nome do projeto* de sua escolha. Agora, será executado o comando acima através do prompt.

Aqui, o nome do projeto será  *angular-alpha*.

A próxima pergunta é sobre a folha de estilo: s opções disponíveis são CSS, Sass, Less e Stylus. Na captura de tela acima, a seta está no CSS. Para alterar, você pode usar as teclas de seta para selecionar a necessária para a configuração do seu projeto. No momento, discutiremos o CSS para a configuração do projeto.

```
npm
Microsoft Windows [versão 10.0.18363.1734]
(c) 2019 Microsoft Corporation. Todos os direitos reservados.

C:\Users\Black Yellow Box>ng new angular-alpha
? Would you like to add Angular routing? Yes
? Which stylesheet format would you like to use? (Use arrow keys)
> CSS
  SCSS   [ https://sass-lang.com/documentation/syntax#scss                ]
  Sass   [ https://sass-lang.com/documentation/syntax#the-indented-syntax ]
  Less   [ http://lesscss.org                                             ]
```

A próxima pergunta será a seguinte:

```
? Do you want to enable Server-Side Rendering (SSR) and Static Site Generation (SSG/Prerendering)? (y/N) N
```

basta digitar a letra N (NO).

O projeto *angular-alpha* foi criado com sucesso. Ele instala todos os pacotes necessários para que nosso projeto seja executado no Angular. Vamos agora entrarna pasta do projeto criado, que está no diretório ***angular-alpha.***

Altere o diretório na linha de comando usando a linha de código fornecida:

```
cd angular-alpha
```

**Visual Studio Code**

Este é um IDE de código aberto do Visual Studio. Está disponível para plataformas Mac OS X, Linux e Windows. O VScode está disponível em
- https://code.visualstudio.com/

Instalação no Windows - se ainda não estive instalado em seu ambiente, basta seguir as mesmas estapas da instalação anterior (para typescript).

Não foi iniciado nenhum projeto nele. É preciso trazer o projeto criado - usando o angular-cli e seu respectivo comando para criação – para dentro da IDE Visual Studio Code. O procedimento para executar essa tarefa é o seguinte:

Clique em *File* > ao clicar em File no menu de tarefas no topo da IDE se abrirá uma janela de contexto: ao abri a janela selecione >*Open Folder*. Na sequencia, encontre o local onde seu projeto Angular foi gerado; clique na pasta que contem seu projeto e, dessa forma ele será importado para dentro da IDE.

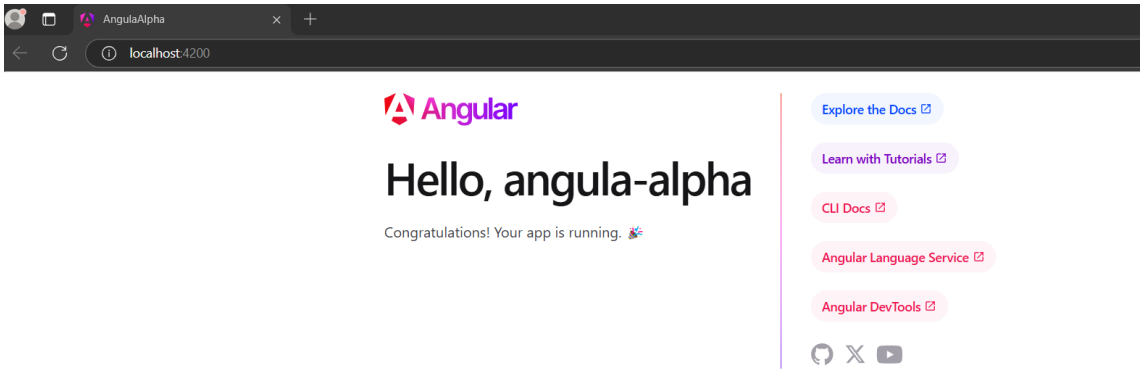
Vamos abrir o ***angular-alpha*** e ver como é a estrutura da pasta.

Agora que o projeto está disponível na IDE, é possível compila-lo com o seguinte comando:

```
ng serve --open
```

O comando ng serve cria o aplicativo e inicia o servidor web:

O servidor web inicia na porta 4200. Depois de executar o URL, ***http://localhost: 4200/*** no navegador, ocorrerá um direcionamento para a seguinte tela:




A pasta angular-alpha / tem a seguinte **estrutura de pastas:**

- **node_modules /** - O pacote npm instalado é node_modules. Você pode abrir a pasta e ver os pacotes disponíveis.
- **src /** - Esta pasta é onde iremos trabalhar no projeto usando o Angular. Especialmente usando os arquivos localizados dentro da pasta *src / app* que foi criada durante a instalação do projeto e contém todos os arquivos necessários para trabalharmos no *angular-alpha*.

A pasta angular-alpha contém alguns arquivos em sua estrutura:

- **editorconfig** - Este é o arquivo de configuração do editor.

- **.gitignore** - Um arquivo .gitignore deve ser confirmado no repositório, para compartilhar as regras de ignorar com outros usuários que clonam o repositório.
- **angular.json** - basicamente contém o nome do projeto, a versão do cli etc.
- **package.json** - O arquivo package.json informa quais bibliotecas serão instaladas no node_modules quando você executar o npm install. Caso você precise adicionar mais bibliotecas, adicione-as no arquivo package.json e execute o comando npm install.
- **tsconfig.json** - basicamente contém as opções do compilador necessárias durante a compilação.
- **tslint.json** - Este é o arquivo de configuração com regras a serem consideradas durante a compilação.

A pasta **src /** é a pasta principal, que possui internamente uma estrutura de arquivos diferente.

**Pasta App**

Ela contém os arquivos descritos abaixo. Esses arquivos são instalados pelo angular-cli por padrão:

**app.routes.ts**

Este arquivo tratará do roteamento necessário para o seu projeto. Ele está conectado ao módulo principal, ou seja, *app.module.ts*.

Observe a estutura do arquivo, abaixo:

```
import { Routes } from '@angular/router';

export const routes: Routes = [];
```

**app.component.html**

O código html que será exibido em tela, geralmente, estará disponível neste arquivo.

**app.component.spec.ts**

Esses são arquivos gerados automaticamente que contêm testes de unidade para o componente de origem.

**app.component.ts**

A classe (estrutura lógica) para as regaras de funcionamento do componente é definida neste arquivo. Você pode fazer o processamento da

estrutura html no arquivo *.ts*. O processamento incluirá atividades como conectar-se ao banco de dados, interagir com outros componentes, roteamento, services, etc.

Observe a estrutura de código abaixo:

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet],
  templateUrl: './app.component.html',
  styleUrl: './app.component.css'
})
export class AppComponent {
  title = 'angula-alpha';
}
```

**index.html**

Este é o arquivo que é exibido no navegador.

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>AngularAlpha</title>
  <base href="/">
  <meta name="viewport" content="width=device-
width, initial-scale=1">
  <link rel="icon" type="image/x-icon" href="favicon.ico">
</head>
<body>
  <app-root></app-root>
</body>
</html>
```

O corpo tem ***<app-root></app-root>*** . Esse é o seletor usado no arquivo ***app.component.ts*** e exibirá os detalhes do arquivo ***app.component.html*** .

**main.ts**

*main.ts* é o arquivo de onde começamos o desenvolvimento do projeto. Começa com a importação do módulo básico que precisamos. No

momento, se você ver angular / core, angular / plataforma-dinâmica do navegador, app.module e ambiente, é importado por padrão durante a instalação do angular-cli e a configuração do projeto.

```
import { enableProdMode } from '@angular/core';
import { platformBrowserDynamic } from '@angular/platform-
browser-dynamic';

import { AppModule } from './app/app.module';
import { environment } from './environments/environment';

if (environment.production) {
  enableProdMode();
}

platformBrowserDynamic().bootstrapModule(AppModule)
  .catch(err => console.error(err));
```

O *platformBrowserDynamic(). BootstrapModule(AppModule)* possui a referência do módulo pai AppModule. Portanto, quando é executado no navegador, o arquivo é chamado index.html. Index.html refere-se internamente a main.ts que chama o módulo pai, ou seja, AppModule quando o código a seguir é executado:

```
platformBrowserDynamic().bootstrapModule(AppModule).catc
h(err => console.error(err));
```

Quando o AppModule é chamado, ele chama app.module.ts, que chama mais o AppComponent com base na autoinicialização da seguinte maneira:

```
bootstrap: [AppComponent]
```

**styles.css**

Este é o arquivo de estilo necessário para o projeto.

**tsconfig.app.json**

Isso é usado durante a compilação, possui os detalhes de configuração que precisam ser usados para executar o aplicativo.

**tsconfig.spec.json**

Isso ajuda a manter os detalhes para o teste.

A estrutura final do arquivo será a seguinte:

No **app.component.ts** , existe um seletor: **app-root,** usado no arquivo index.html. Isso exibirá o conteúdo presente em **app.component.html** .

## Data Binding

O **Data Binding** no **Angular** é um conceito central no framework que permite a troca de dados entre o componente e a interface do usuário de forma eficiente e reativa. Esse processo envolve a sincronização entre os dados da aplicação e a visualização dos dados na interface, proporcionando uma interação dinâmica entre a lógica da aplicação e os elementos da interface.

### Definição

No Angular, o **Data Binding** é uma técnica que conecta a parte lógica da aplicação (componente) com a interface (view). Existem diferentes tipos de data binding, incluindo **interpolation**, **property binding**, **event binding** e **two-way binding**, cada um atendendo a diferentes necessidades de comunicação entre a view e o componente.

- **Interpolation**: Permite que expressões sejam incorporadas dentro do template HTML. Exemplo: {{ expression }}.
- **Property Binding**: Ligação entre o valor de uma propriedade do componente e a propriedade de um elemento HTML.
- **Event Binding**: Permite que eventos do usuário, como cliques e digitação, sejam capturados e usados para invocar métodos no componente.
- **Two-way Binding**: Combina property e event binding, permitindo que os dados fluam bidirecionalmente entre a view e o componente.

### Aplicação e Benefícios

Segundo estudos, o uso eficiente do Data Binding no Angular reduz a complexidade do gerenciamento de estados e torna o desenvolvimento mais ágil, visto que a manipulação dos dados da interface é automatizada e eficiente (Haddad, 2021). Isso permite que o desenvolvimento de aplicações interativas seja mais intuitivo, sem a necessidade de gerenciamento explícito da interface

do usuário. Ao usar o **two-way binding**, por exemplo, o Angular automaticamente reflete mudanças de valor na interface e vice-versa, melhorando a responsividade e a usabilidade da aplicação (Tombari & Serang, 2019).

O **Data Binding** também é fundamental para garantir a separação de responsabilidades entre a lógica do componente e a renderização da view, seguindo os princípios do **MVVM (Model-View-ViewModel)**, uma das abordagens de design adotadas pelo Angular. A abordagem MVVM é uma das mais usadas em sistemas de desenvolvimento modernos, promovendo um design desacoplado e fácil manutenção (Alaa, 2020).

## Interpolação (Interpolation)

A ligação de dados de interpolação (interpolation) é a maneira mais popular e mais fácil de ligação de dados no Angular. Esse recurso também está disponível nas versões anteriores da estrutura Angular. Na verdade, o **contexto entre chaves** é a expressão do modelo (model) que o Angular avalia primeiro e depois converte em sequências de caracteres (string). A interpolação (interpolation) usa a expressão de chaves, ou seja, {{}} para renderizar o valor associado ao componente (component). Pode ser uma sequência estática, valor numérico ou um objeto do seu modelo (model) de dados. No

Angular, usamos assim: {{firstName}}. O exemplo abaixo mostra como podemos usar a interpolação no componente para exibir dados no front end.

```
<span>{{ title }} aplicativo angular funcionando!</span>
```

**Implementação Interpolation (Interpolação)**

Neste passo, vamos implementar cada um dos data binding estudados até este momento. No exemplo abaixo, faremos uso da Interpolação (Interpolation) no Angular para diferentes tipos de dados.

Crie um novo componente e nomeie-o como ***interpolation*** dentro da pasta app do seu projeto – para criar este novo componente - via angular cli – é necessário usar o comando abaixo:

***ng generate component interpolation***

Abra a pasta do componente e edite os arquivos conforme indicado abaixo:

***interpolation.component.ts***

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-interpolation',
  templateUrl: './interpolation.component.html',
  styleUrls: ['./interpolation.component.css']
})
export class InterpolationComponent{
// criando as propriedades para data binding interpolation
public x: number = 10
public umArray: Array<number> = [10, 22, 14]
public dataHoje: Date = new Date()

public statusBooleano: boolean = true

public exibirTexto(): string{
  return 'Texto retornado a partir de uma função!'
}
}
```

***interpolation.component.html***

```
<!-- vinculando as propriedades descritas no ts -->
<div>
    <h1>Interpolation Data Binding</h1>

    <span>O valor da variavel x é: {{ x }}</span>
    <br /><br />

    <span>O array de números é copmposto pelos seguintes va
lores: {{ umArray }}</span>
    <br /><br />

    <span>A data de hoje é: {{ dataHoje }}</span>
    <br /><br />
```

```
    <span>O status booleano indicado na variavel é: {{ stat
usBooleano }}</span>
    <br /><br />

    <span>{{ statusBooleano ? "Este é o texto para o status
 True" : "Este é o texto para o status False!" }}</span>
    <br /><br />
    <span>{{ exibirTexto() }}</span>
    <br /><br />
</div>
```

Antes de obervar o resultado da primeira implementa, no browser, navegue até o arquivo ***app.routes.ts*** e implemente o código abaixo com se segue. Neste arqruivo serão craidas todas as rotas para todos os componentes deste projeto.

***Obs.: este passo deve ser repetido para todos os novos componentes criados dentro deste projeto***

## Routing

Routing (Rotear) significa basicamente navegar entre as páginas. Aqui as páginas às quais existem referencias estarão na forma de componentes.

O arquivo responsável pelo roteamento(criação de rotas para os componentes) é  arquivo routes.ts Os detalhes do arquivo  estão indicados abaixo:

***app.routes.ts***

```
import { Routes } from '@angular/router';

export const routes: Routes = [];
```

Aqui, é preciso observar que esse arquivo é gerado por padrão quando a aplicação foi criada. Se não forem adicionados, deverá ser criado manualmente.

Portanto, no arquivo acima, foram importadas as dependencias *Routes* e *Routes* de *@angular/router*.

Há uma *const routes* definida, que é do tipo *Routes*. É um array onde serão implementadas todas as rotas dos componentes deste projeto.

As *const routes é* fornecida ao *RouterModule* como mostrado em *@NgModule*. Para exibir os detalhes de roteamento para o usuário, é preciso adicionar a diretiva ***<router-outlet></router-outlet>*** onde queremos que a exibição seja mostrada – por padrão, as tags são referenciadas no arquivo *app.componet.html* - a view principal do projeto - como mostrado abaixo:

```
<router-outlet></router-outlet>
```

Agora, a partir da criação do componente ***Interpolation***, crie um novo compoentne chamado ***Home*** – seguindo os mesmos passo realizados para criar o componente ***Interpolation*** – este novo componente servirá de base para o projeto. Na seqeuncia, crie as rotas necessárias para que eles sejam acessados e o resultado da implementação seja exibido no browser. Implemente o código abaxio com se segue:

***app.routes.ts***

```
import { Routes } from '@angular/router';
// abaixo, serão importados todos os componentes para os
quais desejamos estabelecer rotas
import { InterpolationComponent } from
'./componentes/interpolation/interpolation.component';

export const routes: Routes = [
    // aqui, serão compostas todas as rotas para os
componentes do projeto
    // http://localhost:4200/interpolation
    {path: 'interpolation', component:
InterpolationComponent}

];
```

Mais um passo precisa ser dado. Navegue até o arquivo ***app.component.html*** e crie os routerLinks necessários para, através de um clique no link, seja possível exibir o componente no browser. Implemente o código abaixo como se segue:

***app.component.html***

```
<div class="right-side">
     <div class = "pill-group">
      <a class = "pill" routerLink = "/interpolation">Interpolação</a>
</div>

<router-outlet />
```

Agora, navegue até o arquivo *styles.css* e implemente o bloco de código a seguir:

***styles.css***

```
/* You can add global styles to this file, and also import
other style files */
.content {
    display: flex;
    margin: 82px auto 32px;
    padding: 0 16px;
    max-width: 960px;
    flex-direction: column;
    align-items: center;
  }
```

Ao final destes passos, o resultado será semelhante ao indicado abaixo:

Finalmente, é possível clicar no link Interpolação e exibir o resultado da implementação feita no componente. Observe o resultado. Salve o projeto e observe a saída.

## Ligação/Vinculação de dados baseada em propriedades (Property Binding)

No Angular, existe outro mecanismo de ligação, chamado de Vinculação de Propriedade (property binding). Em sua essência, vinculação/ligação de

dados baseada em propriedade é exatamente o mesmo que interpolação. É também chamado de ligação unidirecional. Para ligação de propriedade (property binding) é usado [ ] para enviar os dados do componente para o modelo HTML. A maneira mais comum de usar o property binding é atribuir qualquer propriedade da tag do elemento HTML ao [ ] com o valor da propriedade do componente; observe o snnipet abaixo:

```
Exibir valor com property-
binding : <input [value]="umvalor" />
```

**Implementação property Binding**

Nesta sequencia, vamos implementar o data binding - conforme exemplo abaixo – baseado em propriedade (property binding). Crie um novo componente e nomeie-o como ***property-binding*** dentro da pasta app do seu projeto – para criar este novo componente - via angular cli – é necessário usar o comando abaixo:

***ng generate component property-binding***

Abra a pasta do componente e edite os arquivos conforme indicado abaixo:

***property-binding.component.ts***

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-p-binding',
  standalone: true,
  imports: [],
  templateUrl: './p-binding.component.html',
  styleUrl: './p-binding.component.css'
})
export class PBindingComponent {
  // definir o titulo do componente
  tituloComp: string = 'Implementação Property binding'

  // definir uma nova prop para vincular na view
  umTexto: string = 'Texto da prop.'

}
```

***property-binding.component.html***

```
<div>
    <h2>{{ tituloComp }}</h2>

    <span>
        <input [value] = "umTexto" />
    </span
</div>
```

***app.component.html***

```
<div class="right-side">
     <div class = "pill-group">
      <a class = "pill" routerLink =
"/interpolation">Interpolação</a>
      <a class = "pill" routerLink = "/p-
binding">PBinding</a>
</div>
```

## Vinculação/ligação por evento (Event Binding)

A ligação de eventos (event binding) é outra das técnicas de ligação de dados (data binding) disponíveis no Angular. Essa técnica não funciona com o valor dos elementos da interface do usuário - funciona com as **atividades de eventos dos elementos da interface do usuário**, como **evento de clique**, **evento de desfoque** (blur-event) etc. Nas versões anteriores do Angular, são usadas diferentes tipos de diretivas (directives). como ng-click, ng-blur para vincular qualquer ação de evento específica de um controle (control) HTML. Porém, na versão Angular atual, precisamos usar a mesma propriedade do

elemento HTML (como clicar, alterar etc.) e usá-la entre parênteses. No Angular, para propriedades, usamos colchetes e, em eventos, parênteses.

```
   <div>

    <h2>Exemplo Event Binding Angular</h2>
    <input type="button" value="Click" (click)="exibirAlert
a()" />
    <br /><br />
    <input type="button" value="Mouse Enter" (mouseenter)="
exibirAlerta()" />
</div>
```

**Implementação Event Binding (Ligação/Vinculação de dados baseada em evento)**

Agora, vamos implementar o data binding - conforme exemplo abaixo – baseado em evento (event binding). Crie um novo componente e nomeie-o como ***e-binding*** dentro da pasta app do seu projeto – para criar este novo componente - via angular cli – é necessário usar o comando abaixo:

***ng generate component e-binding***

Abra a pasta do componente e edite os arquivos conforme indicado abaixo:

***e-binding.component.ts***

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-e-binding',
  standalone: true,
  imports: [],
  templateUrl: './e-binding.component.html',
  styleUrl: './e-binding.component.css'
})
export class EBindingComponent {
  // definir o titulo do componente
  tituloComp: string = 'Implementação Event binding'

  // definir um método que será chamado à sua execução quando um
evento for disparado - a partir da view
```

```
  // este binding determina que, na view, deva constar a chamada
de determinado método.
  exibirAlerta(): void{
    // uso da função auxiliar log()
    console.log('Evento disparado...')

    // a informação, configurada abaixo, aparecerá numa janela
de alerta  - na view
    alert('Um evento - a partir da view - foi disparado.')
  }

}
```

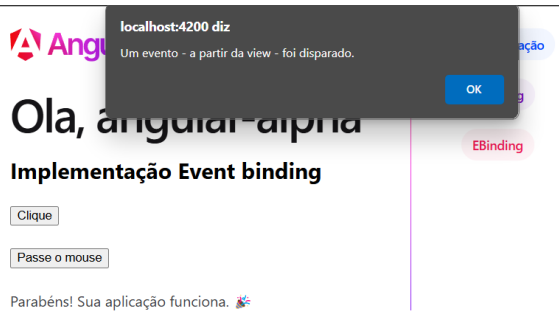
***e-binding.component.html***

```
<div>
    <h2>{{ tituloComp }}</h2>

    <input type = "button" value = "Clique" (click) =
"exibirAlerta()" />
    <br />
    <br />
    <input type = "button" value = "Passe o mouse" (mouseenter)
= "exibirAlerta()" />
</div>
```

***app.component.html***

```
<div class="right-side">
     <div class = "pill-group">
      <a class = "pill" routerLink =
"/interpolation">Interpolação</a>
      <a class = "pill" routerLink = "/p-binding">PBinding</a>
      <a class = "pill" routerLink = "/e-
binding">EBinding</a>
    </div>
</div>
```

A saída será a seguinte:

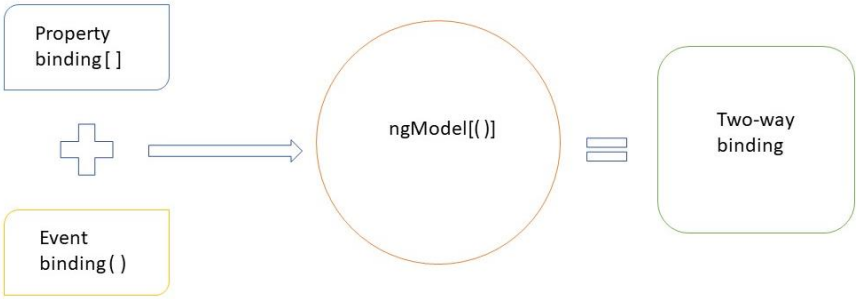



## Vinculação/ligação de dados bidirecional (Two-Way Data Binding)

No Angular Framework, as técnicas de ligação de dados (data binding) mais usadas e importantes são conhecidas como Vinculação de Dados Bidirecional (two-way data binding). A ligação bidirecional é usada principalmente no campo do tipo de entrada ou em qualquer elemento de formulário em que o usuário possa fornecer valores de entrada do navegador ou fornecer qualquer valor ou, ainda, alterar qualquer valor de controle por meio do navegador. Por outro lado, o mesmo é atualizado automaticamente nas variáveis do componente e vice-versa. Da mesma forma, no Angular, temos uma diretiva chamada ngModel, e ela precisa ser usada como abaixo:

```
  <div>
    <div>
        <span>Informe seu nome </span>
        <input [(ngModel)]="x" type="text"/>
    </div>
    <div>
        <span>Seu nome é: </span>
        <span>{{ x }}</span>
    </div>
</div>
```

Usamos [ ], pois na verdade é uma ligação de propriedade (property binding), e parênteses são usados para o conceito de ligação de evento (event binding), ou seja, a notação de ligação de dados bidirecional é [()].




ngModel executa a ligação de propriedades (property binding) e a ligação de eventos (event binding). Na verdade, a ligação de propriedade (property binding) do ngModel (ou seja, [ngModel]) realiza a atividade para atualizar o elemento de entrada com um valor. Enquanto que (ngModel) (evento (ngModelChange)) instrui o “mundo externo” quando qualquer alteração ocorreu no elemento DOM.

**ImplementaçãoTwo Way Data Binding (Vinculação/Ligação de dados bidericional)**

Agora, nesse exemplo, mostraremos como usar a ligação de dados bidirecional em angular. Para esse fim, crie um novo componente e nomeie-o como ***two-way*** dentro da pasta app do seu projeto – para criar este novo componente - via angular cli – é necessário usar o comando abaixo:

```
ng generate component two-way
```

Abra a pasta do componente e edite os arquivos conforme indicado abaixo:

Quando a diretiva *ngModel* ou ligação de dados bidirecional em nossos componentes são implementados, é necessário incluir o módulo de

dependencia *FormsModule* do Angular no arquivo *two-way.component.ts*, como abaixo - o *ngModel* não funcionará sem a inclusão do *FormsModule:*

***two-way.component.ts***

```
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';

@Component({
  selector: 'app-two-way',
  standalone: true,
  imports: [FormsModule],
  templateUrl: './two-way.component.html',
  styleUrl: './two-way.component.css'
})
export class TwoWayComponent {

  // definir o titulo do componente
  tituloComp: string = 'Implementação Two-way binding'

  // estabelecer a prop que será vinculada na view
  algumValor: string = ''

}
```

***two-way.component.html***

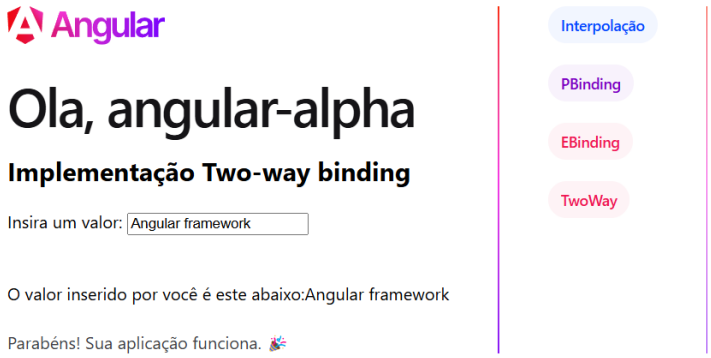
```
<div>
    <h2>{{ tituloComp }}</h2>

    <div>
        <span>Insira um valor: </span>
        <input type = "text" [(ngModel)] = "algumValor" />
    </div>
    <br /><br />

    <div>
        <span>O valor inserido por você é este abaixo:</span>
        <span>{{ algumValor }}</span>
    </div>
</div>
```

***app.component.html***

```
<div class="right-side">
    <div class = "pill-group">
```

```
      <a class = "pill" routerLink = "/interpolation">Interpolação</a>
      <a class = "pill" routerLink = "/p-binding">PBinding</a>
      <a class = "pill" routerLink = "/e-binding">EBinding</a>
      <a class = "pill" routerLink = "/two-way">TwoWay</a>
    </div>
</div>
```

A saída é a seguinte:




## Diretivas

As *Directives Angular* proporcionam a manipulação do DOM. Fazendo das direcitives é possível alterar aparência, comportamento ou a estrutura do layout de um elemento DOM.

Existem três tipos de directives no Angular:

1. Diretivas de estrutura (Structural Directives)
2. Diretivas de atributo (Attribute Directives)
3. Diretaivas de componentes ou customizadas (Directive Component)

### Diretivas de estrutura

As directives estruturais (structural directives) podem alterar o layout do DOM adicionando e removendo elementos DOM. Todas as directives estruturais são precedidas pelo símbolo asterisco (*).

### *ngIf

A direitva *ngIf é uma das diretivas Angular estrutural mais usadas; fazendo uso dessa diretiva é possível incluri/excluir elementos DOM

considerando alguma condição para tomada de decisão. Seu uso parte da premissa de acoplamento da diretiva a um elemento DOM – pode ser adicionada a qualquer elemento DOM como *div, p, h1, seletor de componente*, entre outros. Como qualquer outra diretiva estrutural, é prefixado com *(asterisco)

A descrição como se segue - *ngIf - é vinculada a uma expressão condicional. Dessa forma, a expressão é, então, avaliada pela diretiva. O retorno da verificação deve ser *"true*" ou "*false".*Se a expressão for avaliada como *false*, o Angular remove o elemento inteiro do DOM. Se *true*, o elemento será inserido no DOM.

Usando o ***NOT lógico*** – caracter exclamação *(!)* -, é possível criar uma imitação da condição *else*. Observe o snnipet abaixo:

```
<p *ngIf="!condicao">
    conteúdo exibido quando a condição for false
</p>
```

**condição** - aqui, a condição pode ser qualquer coisa; uma propriedade da classe do componente – por exemplo; também um método na classe do componente. Mas deve ser avaliado como true/false. A diretiva *ngIf* tentará forçar o valor para verificação booleana.

Considerando este cenário, a implementação adequada é usar o bloco else – como opção.

**ngIf else**

ngIf possibilita a definição do bloco else - como opção - usando o ***ng-template;*** a documentação oficial define ng-template como: *"a diretiva ng-template representa um template Angular: isso significa que o conteúdo desta tag irá conter parte de um template (estrutura de view), que pode então ser composto junto com outros templates para formar o template do componente final."*

A expressão algoritmica começa com uma condição seguida por um ponto e vírgula. Na sequencia, observa-se a implementação da instrução *else* acoplada a um template denominado *blocoElse*. O *template* pode ser definido

em qualquer lugar do bloco de código acima, usando a diretiva *ng-template*. Uma boa prática é indica-lo logo após a implementação da diretiva *ngIf* facilitando, assim, a leitura do código implementado.

Quando a condição é avaliada como *false*, a diretiva *ng-template* com o nome *#blocoElse* é, então, processada pela diretiva *ngIf.*

**ngIf then else**

É possível, também, definir o bloco *then else* usando o *ng-template*. Quando a condição é *true*, o *template blocoThen* é exibido. Caso a condição seja avaliada como *false,* o *template blocoElse* é exibido.

**Atributo hidden**

Observe o snnipet abaixo:

```
<p [hidden]="condição">
    Algum conteúdo a ser exibido
</p>
```

Considerando o uso da diretiva *ngIf* – em relação a propriedade hidden observa-se a seguinte definição: *nIf* não oculta o elemento DOM. Ele remove o elemento inteiro junto com sua sub-árvore; também remove o estado correspondente, liberando os recursos anexados ao elemento; o atributo *hidden* não remove o elemento do DOM, apenas esconde. Essa é a direferença entre *[hidden]='false'* e **ngIf='false'*; o primeiro método simplesmente oculta o elemento. O segundo método *ngIf* remove o elemento completamente do DOM.

**Implementação**

Agora, crie um novo projeto chamado *directive* ou continue usando o projeto angular-alpha para implementar a diretiva *ngIf*.

Para esse fim, crie um novo componente e nomeie-o como ***dir-if*** dentro da pasta app do seu projeto – para criar este novo componente - via angular cli – é necessário usar o comando abaixo:

```
ng generate component dir-if
```

Abra a pasta do componente e edite os arquivos conforme indicado abaixo:

***dir-if.component.ts***

```
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';

@Component({
  selector: 'app-dir-if',
  standalone: true,
  imports: [FormsModule],
  templateUrl: './dir-if.component.html',
  styleUrl: './dir-if.component.css'
})

export class DirIfComponent {
  tituloComp: string = 'Implementação @if/ *ngIf'

  idade: number = 18

}
```

***dir-if.component.html***

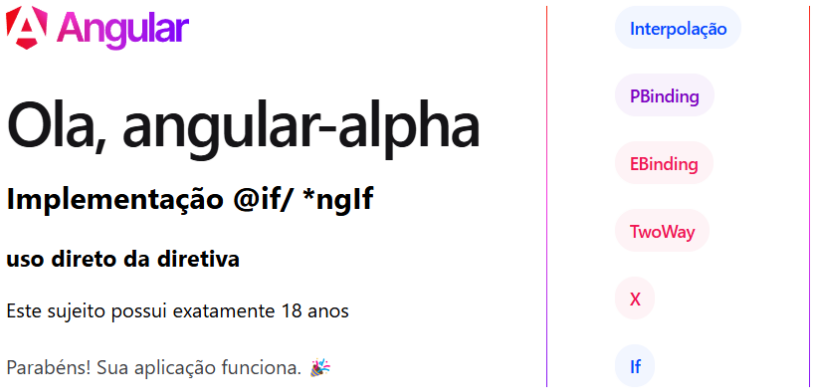
```
<div>
    <h2>{{ tituloComp }}</h2>

    <!-- uso direto da diretiva @if/else-->
    <h3>uso direto da diretiva</h3>

    <!--
    <p *ngIf(idade > 18; then OutrMiniView else MiniView)>
       -- -este conteudo nunca aparecerá
    </p>
    <ng-template #MiniView>
        hgfkjfhglkjsdfgljkghkjlfhgkl
    </ng-template>

    <ng-template #OutraMiniView>
         o cara é de maior
    </ng-template>

    -->
    <!-- uso da diretiva @if/else if/else-->
    @if(idade > 18){
        <p>
            Este sujeito possui mais de 18 anos
        </p>
```

```
    }@else if(idade == 18){
        <p>
            Este sujeito possui exatamente 18 anos
        </p>
    }@else{
        <p>
            Esse sujeito possui menos que 18 anos
        </p>
    }
</div>
```

***app.component.html***

```
    <div class="right-side">
      <div class = "pill-group">
      <a class = "pill" routerLink = "/interpolation">Interpolação</a>
      <a class = "pill" routerLink = "/p-binding">PBinding</a>
      <a class = "pill" routerLink = "/e-binding">EBinding</a>
      <a class = "pill" routerLink = "/two-way">TwoWay</a>
      <a class = "pill" routerLink = "/x">X</a>
      <a class = "pill" routerLink = "/dir-if">If</a>
      </div>
    </div>
```

O resultado no browser deve ser semelhante ao indicado abaixo.Com o checkbox desativado:




**ngFor**

O ngFor é uma *structural directive Angular*, que repete uma parte do template HTML uma vez por cada item de uma lista iterável (coleção); observe o snnipet abaixo:

```
<tr *ngFor="let <item> of <itens>;">
    <td>{{item.itens}}</td>
</tr>
```

*<tr></tr>* - é o elemento sobre o qual a diretiva *ngFor* é aplicada. Este elemento *<tr></tr>* será repetido para cada item encontrado dentro da coleção.

**ngFor* – é a sintaxe que inidica o uso da diretiva. O elemento * *(astersico)* representa sua sintaxe como diretiva estrutural.

*let <item>of<itens>* - *item* é a variável auxiliar (de contagem/iteração) do template. Representa o *elemento* atualmente iterado da coleção *<itens>*, que deve ser exibida ao usuário. Normalmente a coleção é arrya de elementos.

O escopo da variavel *item* está dentro do *<td></td>.* Você pode acessá-lo em qualquer lugar dentro dele (*<td></td>*), mas não fora dele.

**Implementação**

Agora, crie um novo projeto chamado directive ou continue usando o projeto angular-alpha para implementar a diretiva ngFor.

Para esse fim, crie um novo componente e nomeie-o como ***dir-for*** dentro da pasta app do seu projeto – para criar este novo componente - via angular cli – é necessário usar o comando abaixo:

```
ng generate component dir-for
```

Após criar o novo componente é importante criar uma nova pasta dentro do projeto – esta pasta será criada, também, dentro da pasta app do projeto – use o comando `mkdir model` ou clique com o botão direito do mouse e crie a nova pasta com o nome ***model***. Dentro desta pasta, crie dois novos arquivos – typescript - com o nome ***filme.ts*** e outro com o nome ***colunas.ts.*** Estes arquivos trabalharão como o models domain para o conjunto de dados que será manipulado pelo novo componente. Observe os códigos abaixo:

***filme.ts***

```
export class Filme {
    // definir os fields/atributos/.... que compõem o model
    titulo:string
    direcao: string
    elenco: string
    anoLancamento: number

    constructor(
        titulo: string,
        direcao: string,
        elenco: string,
        anoLancamento: number
    ){
        // acessar as props da classe e atribui-las com valores
de argumento que forma associados aos parametros do construtor
        this.titulo = titulo
        this.direcao = direcao
        this.elenco = elenco
        this.anoLancamento = anoLancamento
    }
}
```

***colunas.ts***

```
// este é o model domain para, posteriormente, criar as props
que irão nomear as colunas  da futura tabela para exibição dos
dados
export class Colunas {
    coluna1: string
    coluna2: string
    coluna3: string
    coluna4: string

    //definir o construtor da classe
    constructor(
        coluna1: string,
        coluna2: string,
        coluna3: string,
        coluna4: string
    ){
        this.coluna1 = coluna1
        this.coluna2 = coluna2
        this.coluna3 = coluna3
        this.coluna4 = coluna4
    }
}
```

Neste próximo passo, é importante criar uma nova pasta dentro do projeto – esta pasta será criada, também, dentro da pasta app do projeto – use o comando *ng generate class fonteDados/dadosCompFor* – este comando criará a nova past e o respectivo arquivo; ou clique com o botão direito do mouse e crie a nova pasta com o nome ***fonteDados***. Dentro desta pasta, crie o novo arquivos – typescript - com o nome ***dadosCompFor.ts*** Este arquivo trabalhará como uma "fonte de dados" para a manipulação através do novo componente. Observe os códigos abaixo:

dadosCompFor.ts

```
// este arquivo será nossa fonte de dados

import { Filme } from '../models/filme';
import { Colunas } from '../models/colunas';

export const dadosFilmes: Filme[] = [

    new Filme('Um Estranho no Ninho', 'Milos Form', 'Jack
Nicholson e outros', 1976),
    new Filme('Forest Gump - o contador de historias', 'Robert
Zemeckis', 'Tom Hanks e outros', 1994),
    new Filme('Laranja Mecanica', 'Stanley Kubrik', 'Macolm
Mcdowel e outros', 1971),
    new Filme('A Lista Schindler', 'Steven Spielberg', 'Liam
Neeson e outros', 1993),
    new Filme('O poderoso Chefão', 'Francis Ford Copolla',
'Marlon Brando e outros', 1972)
]

// criar o conjunto de dados que dará nome as colunas da nossa
table
export const nomesColunas: Colunas[] = [
    new Colunas('Titulo do Filme', 'Direção', 'Elenco', 'Ano de
Lançamento')
]
```

Agora, abra a pasta do componente e edite os arquivos conforme indicado abaixo:

***dir-for.component.ts***

```
import { Component } from '@angular/core';

// é necessario importar a fonte de dados para que se possa
praticar a vinculação destes mesmos dados na view
import { dadosFilmes, nomesColunas } from
'../../fonteDados/dadosCompFor';

@Component({
  selector: 'app-dir-for',
  standalone: true,
  imports: [],
  templateUrl: './dir-for.component.html',
  styleUrl: './dir-for.component.css'
})
export class DirForComponent {

  tituloComp: string = 'Implementação diretiva @for'
  nomeandoColunas = nomesColunas

  listaDeFilmes = dadosFilmes
  constructor(){
    console.log(this.nomeandoColunas)
    console.log(this.listaDeFilmes)
  }

}
```

Para implementar a diretiva ngFor será necessário:

1. Criar um bloco de elementos HTML, que pode exibir um único filme.
2. Usar o ngFor para repetir o bloco para cada filme.

Navegue até o arquivo *app.component.html* e implemente o código abaixo – como se segue:

***dir-for.component.html***

```
<div>
    <h2>{{ tituloComp }}</h2>

    <!--definição da tabela-->
    <table class="bordaTabela">
        <thead>
```

```
            @for(nomeC of nomeandoColunas; track $index){
            <tr>
                <th>{{ nomeC.coluna1 }}</th>
                <th>{{ nomeC.coluna2 }}</th>
                <th>{{ nomeC.coluna3 }}</th>
                <th>{{ nomeC.coluna4 }}</th>
            </tr>
            }

        </thead>
        <tbody>
            @for(listando of listaDeFilmes; track $index){
            <tr>
                <td>{{ listando.titulo }}</td>
                <td>{{ listando.direcao }}</td>
                <td>{{ listando.elenco }}</td>
                <td>{{ listando.anoLancamento }}</td>
            </tr>
           }

        </tbody>
    </table>
</div>
```

O elemento *<ul></ul>* foi usado para exibir os filmes. O elemento *<li></li>* exibe um único filme. È necessário, então, repetir o elemento*<li></li>* para cada filme. Portanto, aqui, se aplica a diretiva *ngFor - no elemento*<li></li>*.

A instrução *let i of filmes* irá iterar sobre a coleção *filmes,* que é uma propriedade da classe do componente. A variável *i* é a variável auxiliar do *template*, que representa o filme iterado da coleção. Foi usada a vinculação de interpolação angular para exibir o título do filme e o nome do diretor.

***app.componet.html***

```
  <div class="right-side">
    <div class = "pill-group">
      <a class = "pill" routerLink = "/interpolation">Interpolação</a>
      <a class = "pill" routerLink = "/p-binding">PBinding</a>
      <a class = "pill" routerLink = "/e-binding">EBinding</a>
      <a class = "pill" routerLink = "/two-way">TwoWay</a>
      <a class = "pill" routerLink = "/x">X</a>
      <a class = "pill" routerLink = "/dir-if">If</a>
```

```
      <a class = "pill" routerLink = "/dir-for">For</a>

      </div>
    </div>
```

Para melhorar o visual da tabela, insira o código abaixo dentro do arquivo *styles.css*:

***styles.css***

```
/* estilo da table para componentes que usurão os atributos
indicados abaixo*/
.bordaTabela {
    font-family: Arial, Helvetica, sans-serif;
    border-collapse: collapse;
    width: 100%;
   }
  .bordaTabela td, .bordaTabela th {
    border: 1px solid #ddd;
    padding: 8px;
   }
  .bordaTabela tr:nth-child(even){background-color: #f2f2f2;}
   .bordaTabela tr:hover {background-color: #ddd;}
  .bordaTabela th {
    padding-top: 12px;
    padding-bottom: 12px;
    text-align: left;
    background-color: #581845;
    color: white;
   }
```

Execute o projeto e observe o resultado:

Angular

# Ola, angular-alpha

**Implementação diretiva @for**

| Titulo do Filme | Direção | Elenco | Ano de Lançamento |
|---|---|---|---|
| Um Estranho no Ninho | Milos Form | Jack Nicholson e outros | 1976 |
| Forest Gump - o contador de historias | Robert Zemeckis | Tom Hanks e outros | 1994 |
| Laranja Mecanica | Stanley Kubrik | Macolm Mcdowel e outros | 1971 |
| A Lista Schindler | Steven Spielberg | Liam Neeson e outros | 1993 |
| O poderoso Chefão | Francis Ford Copolla | Marlon Brando e outros | 1972 |

Parabéns! Sua aplicação funciona. 🎉

Interpolação

PBinding

EBinding

TwoWay

X

If

For

### Diretivas de atributo

Uma attribute directive ou estilo pode alterar a aparência ou o comportamento de um elemento.

### ngClass

A diretiva *ngClass* é usada para adicionar ou remover as classes CSS de um elemento HTML. Usar *ngClass* pode criar estilos dinâmicos em páginas HTML.

### Implementação

Agora, crie um novo projeto chamado *directive* ou continue usando o projeto angular-alpha para implementar a diretiva *ngClass*.

Para esse fim, crie um novo componente e nomeie-o como ***ng-class*** dentro da pasta app do seu projeto – para criar este novo componente - via angular cli – é necessário usar o comando abaixo:

```
ng generate component dir-ng-class
```

Abra seu projeto e crie uma variável chamada *cssalteradoViaVar* com o seguinte comando: ***ng generate class classes/ng-class-css*** – e implemente o código abaixo, como se segue:

***ng-class-css.ts***

```
export class NgClassCss {
    // definir duas props/atributos....
    cores: boolean = true
    tamanho: boolean = true
}
```

Agora, é necessário implementar as linhas de código abaixo dentro do arquivo .css do compoente. Observe o código abaixo e implemente-o como se segue:

***dir-ng-class.component.css***

```
.cores{
    color: #039C68;
}

.tamanho{
    font-size: 45px;
}
```

Neste passo, implemente a camada lógica do componente, como se segue abaixo:

***dir-ng-class.ts***

```
// Objetivo deste componente: obervar o comportamento da
diretiva ngClass
import { Component } from '@angular/core';

// importar a classe NgClassCss
import { NgClassCss } from '../../classes/ng-class-css';
import { CommonModule } from '@angular/common';

@Component({
  selector: 'app-dir-ng-class',
  standalone: true,
  imports: [CommonModule],
  templateUrl: './dir-ng-class.component.html',
  styleUrl: './dir-ng-class.component.css'
})
export class DirNgClassComponent {
```

```
  // definir o titulo do componente
  tituloComp: string = 'Implementação diretiva ngClass'

  // definir uma prop para vincular na view
  estiloVar: string = 'cores tamanho'

  // praticar o mecanismo de instancia direta com a classe
NgClassCss e gerar um objeto
  objNgClass: NgClassCss = new NgClassCss()
}
```

Implemente o código abaixo na view do componente, como se segue:

***dir-ng-class.component.html***

```
<div>
    <h2>{{ tituloComp }}</h2>
    <!--
        o uso da diretiva ngClass se dá a partir do uso do
property binding
    -->
    <div [ngClass]="'cores tamanho'">
        <!--
            esta vinculação está sendo feita, diretamente, com
as classes css
        -->
        <p>
            Cor e fonte alterados pelas propriedades de estilo
CSS - descritas no arquivo CSS do componente. Definida como uma
string
        </p>
    </div>

    <div [ngClass] = "estiloVar">
        <p>
            Cor e fonte alterados pelas propriedades de estilo
CSS - descritas no arquivo .ts do componente. Definida como uma
prop/var/atributo/field
        </p>
    </div>

    <div [ngClass] = "objNgClass">
        <p>
            Cor e fonte alterados pelas propriedades de estilo
CSS - descritas no arquivo .ts do componente. Definida como um
objeto oriundo de uma classe externa
```

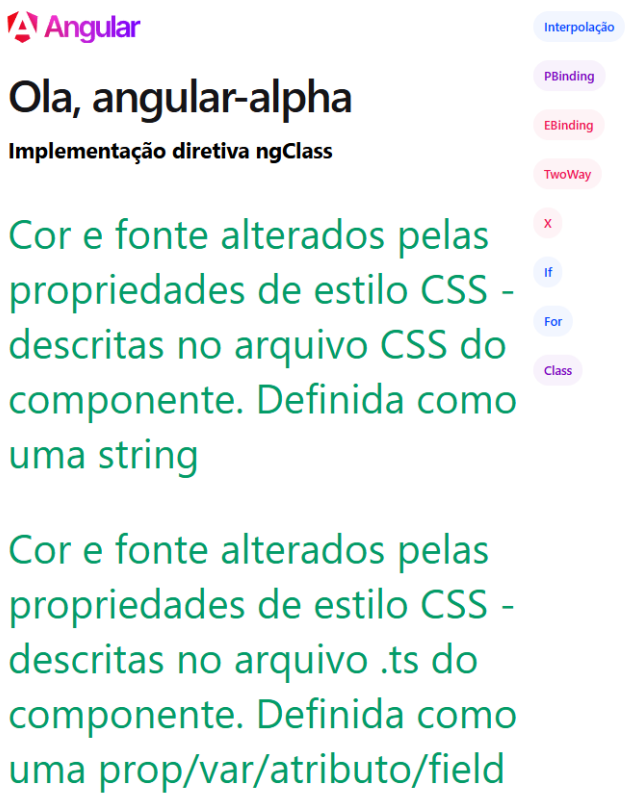
```
        </p>
    </div>
</div>
```

Acima, é possível observar as diferentes formas de implementação das propriedades de estilo configuradas no arquivo *.css*. A diretiva *ngClass* é aplicada em todas as modalidades com diferentes implementações.

***app.component.html***

```
<div class="right-side">
      <div class = "pill-group">
       <a class = "pill" routerLink =
"/interpolation">Interpolação</a>
       <a class = "pill" routerLink = "/p-binding">PBinding</a>
       <a class = "pill" routerLink = "/e-binding">EBinding</a>
       <a class = "pill" routerLink = "/two-way">TwoWay</a>
       <a class = "pill" routerLink = "/x">X</a>
       <a class = "pill" routerLink = "/dir-if">If</a>
       <a class = "pill" routerLink = "/dir-for">For</a>
       <a class = "pill" routerLink = "/ng-class">Class</a>

      </div>
    </div>
```

Salve o projeto. Observe o resultado no browser. Deve ser semelhante ao indicado abaixo:




**ngStyle**

ngStyle é usado para alterar as várias propriedades de estilo de nossos elementos HTML. Também podemos vincular essas propriedades a valores que podem ser atualizados pelo usuário ou por nossos componentes. Observe o *snnipet* abaixo:

```
<div [ngStyle]="{'color': 'blue', 'font-size': '24px',
'font-weight': 'bold'}">
    Algum texto
</div>
```

**Implementação**

Agora, crie um novo projeto chamado *directive* ou continue usando o projeto angular-alpha para implementar a diretiva *ngStyle*.

Para esse fim, crie um novo componente e nomeie-o como ***dir-ng-style*** dentro da pasta app do seu projeto – para criar este novo componente - via angular cli – é necessário usar o comando abaixo:

```
ng generate component dir-ng-style
```

Abra seu projeto e crie uma variável chamada *cssalteradoViaVar* com o seguinte comando: ***ng generate class classes/ng-style-css*** – e implemente o código abaixo, como se segue:

***ng-style-css.ts***

```
export class NgStyleCss {
    //descrever algumas propriedades
    'color': string = '#094468'
    'font-size.%': number = 110
    'font-weight': string = 'bold'
}
```

Abra seu novo componente e, no arquivo ***dir-ng-style.component.ts***, Observe o código abaixo e implemente-o como se segue:

***dir-ng-style.component.ts***

```
import { Component } from '@angular/core';
// importar a classe
import { NgStyleCss } from '../../classes/ng-style-css';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';

@Component({
  selector: 'app-dir-ng-style',
  standalone: true,
  imports: [CommonModule, FormsModule],
  templateUrl: './dir-ng-style.component.html',
  styleUrl: './dir-ng-style.component.css'
})
export class DirNgStyleComponent {
  // definir o titulo do componente
  tituloComp: string = 'Implementação da dirtiva ngStyle'

  // definir algumas props para, posteriormente, vincular na
view
  cores: string = '#380F6C'
  tamanho: number = 20

  // praticar a instancia da classe NgStyleCss(){}
  objStyleCss: NgStyleCss = new NgStyleCss()

}
```

Agora, implemento o código abaixo – como se segue – dentro do arquivo *dir-ng-style.component.html*

***dir-ng-style.component.html***

```
<div>
    <h2>{{ tituloComp }}</h2>
    <br />

    <!-- 1ª implementação da diretiva-->
    <input type = "color" [(ngModel)] = "cores" />
    <div [ngStyle] = "{'color': cores}">
        Mude a cor do texto!
    </div>
    <br />

    <!-- 2ª implementação da diretiva-->
    <input type="number" [(ngModel)] = "tamanho" />
    <div [ngStyle] = "{'font-size.px': tamanho}">
        Mude o tamanho da fonte do texto
    </div>
    <br />

    <!--3ª implementação da diretiva-->
    <div [ngStyle] = "objStyleCss">
        Mudando o tamanho e a cor do texto - via alteração de
props indicadas na classe NgStyleCss
    </div>
</div>
```

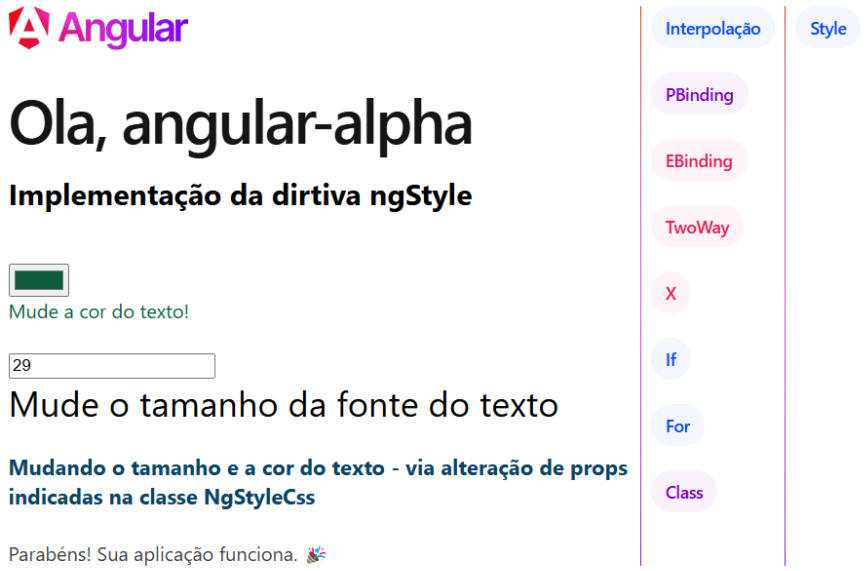
Acima, o código demonstra as diferentes implementações da diretiva ngStyle. Dessa forma é possível alterar o estilo dos componentes do projeto Angular.

***app.component.html***

```
  <!-- estes são os links de navegação entre
componentes/telas-->
    <div class="right-side">
      <div class = "pill-group">

      <a class = "pill" routerLink = "/interpolation">Interpolação</a>
      <a class = "pill" routerLink = "/p-binding">PBinding</a>
      <a class = "pill" routerLink = "/e-binding">EBinding</a>
      <a class = "pill" routerLink = "/two-way">TwoWay</a>
```

```
      <a class = "pill" routerLink = "/x">X</a>
      <a class = "pill" routerLink = "/dir-if">If</a>
      <a class = "pill" routerLink = "/dir-for">For</a>
      <a class = "pill" routerLink = "/ng-class">Class</a>
      </div>
    </div>
  <!-- novo bloco de links-->
    <div class="divider" role="separator" aria-label="Divider"></div>

  <div class="right-side">
      <div class="pill-group">
        <a class="pill" routerLink = "/ng-style">Style</a>
      </div>
    </div>
```

Salve o projeto e execute. O resultado no browser deve ser semelhante ao indicado abaixo:




## Pipes

Neste próximo passo será possível observar o trabalho com *pipes* Angular. Os pipes foram anteriormente chamados de filtros - na primeira versão do angular - e renomeados para pipes – a partir da segunda versão.

O caractere | é usado para aplicar transformações em dados. Observe a sintaxe de um pipe *built-in*:

```
{{ Ola Angular  | lowercase}}
```

É possível trabalhar com pipe aplicando qualquer tido de dado -por exemplo: números inteiros, sequências de caracteres, matrizes, entre outros. Usa-se o caractere | (barra vertical) para converter no formato conforme necessário e exibi-lo no navegador.

Aqui, o objetivo exibir o texto fornecido em maiúsculas. Isso pode ser feito usando pipes da seguinte maneira:

Aqui estão alguns pipes embutidos disponíveis com angular:

- Lowercasepipe
- Uppercasepipe
- Datepipe
- Currencypipe
- Jsonpipe
- Percentpipe
- Decimalpipe
- Slicepipe

Agora, observe a aplicação do pipe.

Para esse fim, crie um novo componente e nomeie-o como ***pipes*** dentro da pasta app do seu projeto – para criar este novo componente - via angular cli – é necessário usar o comando abaixo:

***ng generate component pipes***

Abra a pasta do componente e edite os arquivos conforme indicado abaixo. As seguintes linhas de código nos ajudará a definir as variáveis necessárias no arquivo *pipes.component.ts*:

***pipes.component.ts***

```
import { CommonModule, registerLocaleData } from '@angular/common';

// importar o recurso de localidade especifica
import localePt from '@angular/common/locales/pt';
//importar recursos para a regionalização de moeda - a partir do uso do
pipe currency
import { Component, LOCALE_ID } from '@angular/core';
import { RaizQuadradaPipe } from '../../pipe-custom/raiz-quadrada.pipe';

// referenciar o  metodo registerLocaleData() para registrar ma região
desejada - assim, o service vai atuar sobre a sigla internacional BRL
registerLocaleData(localePt, 'pt')
```

```
@Component({
  selector: 'app-pipes',
  standalone: true,
  imports: [CommonModule, RaizQuadradaPipe],
  providers:[{
    provide: LOCALE_ID,
    useValue: 'pt'
  }],
  templateUrl: './pipes.component.html',
  styleUrl: './pipes.component.css'
})
export class PipesComponent {

  tituloComp: string = 'Implementação Pipes'
  umTextoQualquer: string = 'TEXTO EM MAIUSCULO - PODE SER ALTERADO PARA
MINUSCULO'
  valorFloat: number = 978.55
  umaData: Date = new Date()
  grupoNumeros: number = 123456564.456789

  conjuntoInfos: any = {
    nome: 'Zé Pequeno',
    endereco: {
      rua: 'Rua da Casinnha',
      numero: 1
    }
  }

  calendario: Array<string> = ['Jan', 'Fev', 'Mar', 'Abr', 'Mai', 'Jun',
'Jul', 'Ago',   'Set', 'Out', 'Nov', 'Dez']
  numOutroValor: number = 0.21
}
```

Os pipes serão implementados no arquivo *pipe.component.html*, como mostrado abaixo:

***pipes.component.html***

```
<div>
    <h2>{{ tituloComp }}</h2>

    <table class="bordaTabela">
        <thead>
            <tr>
                <th>Lowercase</th>
                <th>Uppercase</th>
                <th>Currency</th>
                <th>Data</th>
```

```
                <th>Formato numeros</th>
                <th>Json</th>
                <th>Fatiamento</th>
                <th>Porcentagem</th>
            </tr>
        </thead>
        <tbody>
            <td>{{ umTextoQualquer | lowercase }}</td>
            <td>{{ tituloComp | uppercase }}</td>
            <td>
                {{ valorFloat | currency : "BRL" }}
                <br />
                {{ valorFloat | currency: "USD" : "code" : "" :
"en-US"}}
            </td>
            <td>
                {{ umaData | date: "d/M/y" }}
                <br />
                {{ umaData | date: "shortTime" }}
                <br />
                {{ umaData | date: "H:m:s" }}
            </td>
            <td>{{ grupoNumeros | number: "3.1-4" }}</td>
            <td>{{ conjuntoInfos | json }}</td>
            <td>{{ calendario | slice: 2:6 }}</td>
            <td>{{ numOutroValor | percent }}</td>
        </tbody>
    </table>
</div>
```

***app.component.html***

```
  <div class="right-side">
     <div class = "pill-group">
      <a class = "pill" routerLink = "/interpolation">Interpolação</a>
      <a class = "pill" routerLink = "/p-binding">PBinding</a>
      <a class = "pill" routerLink = "/e-binding">EBinding</a>
      <a class = "pill" routerLink = "/two-way">TwoWay</a>
      <a class = "pill" routerLink = "/x">X</a>
      <a class = "pill" routerLink = "/dir-if">If</a>
      <a class = "pill" routerLink = "/dir-for">For</a>
      <a class = "pill" routerLink = "/ng-class">Class</a>
    </div>
    </div>
   <!-- novo bloco de links-->
    <div class="divider" role="separator" aria-label="Divider"></div>
```

```
<div class="right-side">
   <div class="pill-group">
     <a class="pill" routerLink = "/ng-style">Style</a>
     <a class="pill" routerLink = "/dir-custom">DCustom</a>
     <a class="pill" routerLink = "/pipes">Pipes</a>
   </div>
 </div>
```

A captura de tela a seguir mostram a saída para cada pipe:

## Services

Nossos componentes precisam acessar dados (conteúdo ou funcionbalidade). È possivel escrever o código de acesso a dados em cada componente, mas isso, por vezes, é ineficiente e pode quebrar a regra da “responsabilidade única”. O componente deve se concentrar em apresentar dados ao usuário. A tarefa de obter dados do servidor *back-end* deve ser delegada para outra classe. Essa classe é chamada de classe de service (service class). Porque proõe ao service fornecer dados para todos os componentes que precisam.

### O que é um service

O service é um pedaço de código reutilizável com um objetivo focado. Um código que você o usará em muitos componentes em seu aplicativo

**Para que services são utilizados**

1. Recursos independentes de componentes como services de log
2. Compartilhe lógica ou dados entre componentes
3. Encapsula interações externas, como acesso a dados

**Vantagens ao se usar services**

1. Os services são mais fáceis de testar.
2. Os services são mais fáceis de depurar.
3. Você pode reutilizar o service.

**Como criar um service no Angular**

Um *service* Angular é simplesmente uma função Javascript. Tudo o que precisamos fazer é criar uma classe e adicionar métodos e propriedades. Em seguida, podemos criar uma instância dessa classe em nosso componente e chamar seus métodos.

Um dos melhores usos dos *services* é obter os dados de uma fonte interna ou externa. Neste primeiro, será criado um *service* simples, que obtém os dados de produtos e os transmite ao componente.

**Implementação**

Crie um novo projeto chamado *angular-service* (usando o mesmo procedimento para criar o projeto angular-alpha). Usando o projeto *angular-service* para implementar o *Service* será necessário criar um novo arquivo que servirá como model para fazer o tratamento dos dados dispostos na aplicação. Para criar este novo aquivo, clique com o botão direito do mouse na pasta app e crie uma pasta chamada ***model*** – agora, clique com o botão direito do mouse e crie um novo arquivo dentro da pasta *model* do seu projeto – escolha New File e dê o nome ao arquivo de *product.ts*.

```
new file > product.ts
```

**Produto (produtos)**

O arquivo criado ma pasta ***src/app/model*** e nomeado como *produto.ts* deverá conter o código abaixo:

***produto.ts***

```
// criar o model domain para tratar os dados que
// serão operados pelo service
export class Produto{
    // 1ª parte - criar as propriedades - e inicializa-las no
construtor para o acesso e uso do service
    idProduto: number
    nomeProduto: string
    precoProduto: number

    /*
    2ª parte:
     - criar o construtor da classe
     - referenciar/inicializar cada uma das propriedades
indicadas na classe
     - dessa forma as propriedades serão disponibilizadas para o
service e qualquer outro "pedaço" do projeto
    */
   constructor(idProduto: number, nomeProduto: string,
precoProduto: number){
    // inicializar as propriedades/atributos da classe
    this.idProduto = idProduto
    this.nomeProduto = nomeProduto
    this.precoProduto = precoProduto
   }
}
```

A classe *Product* acima é o nosso *domain model* (modelo de domínio).

***Product Service* (Service do produto)**

Em seguida, crie uma nova pasta dentro de *src/app* chamada ***service***. Este *service* angular retornará a lista a partir do *model products*. Para criar o novo service, abra o prompt de comando – via *vs code* ou pelo próprio computador – e, dentro da pasta ***service*** de seu projeto (*angular-alpha*), insira a seguinte instrução:

```
ng g service nomedoservice
ng g service product

CREATE src/app/service/product.service.spec.ts (362
bytes)
CREATE src/app/service/product.service.ts (136 bytes)
```

Após a criação do service, implemente o código abixo como se segue:

***product.service.ts***

```
import { Produto } from './product'

export class ProductService {

 // criar uma função para retornar um lista de produtos
 public getProdutos(){

   // propriedade que será a lista de produtos
   let listaProdutos: Produto[]

   // criando os itens da lista de produtos
   listaProdutos = [
     new Produto(1, 'Quadro Baby Yoda', 199),
     new Produto(2, 'Mascara Darth Vader', 159),
     new Produto(3, 'LightSaber', 89),
      new Produto(4, 'Estrela da Morte Miniatura', 39),
     new Produto(5, 'Boneco Storm Trooper', 59),
     new Produto(6, 'Miniatura Princesa Leia', 119.9),
     new Produto(7, 'Miniatura Han Solo', 259)

   ]
   return listaProdutos
 }
}
```

Primeiro, foi importado para o arquivo *product.service.ts* a class *Product* (a partir de ser arquivo de origem *product.ts*) - criado para estabelecer o model domain.

A classe *ProductService* foi criada e exportada. Foi criado, também, um método chamado *getProdutos()* que retorna a coleção lista*Produtos[ ].* Aqui, o conjunto de dados está disposto dentro da aplicação, no entanto, é possível utilizar procedimento semelhante para obter dados de uma fonte externa a partir do envio de uma solicitação HTTP para uma API de back-end para obter os dados.

O service está pronto. Observe que a classe acima é uma função JavaScript simples. Não há nada angular nisso.

**Invocando o ProductService**

A próxima etapa é chamar o *ProductService* do componente. Abra o arquivo ***app.componente.ts*** do componente e edite-o conforme indicado abaixo.

***app.componet.ts***

```
import { Component } from '@angular/core';
// importar os recursos necessarios para consumir o service
import { ProductService } from '../service/product.service';
import { Produto } from '../model/produto';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']

})
export class AppComponent{
  // casting das propriedades que serão usadas
  cestaProdutos!:Produto[]
  objDoService

  //chamando o construtor
  constructor(){
    this.objDoService = new ProductService()
  }

  // criando uma função para - através do uso do objeto criado
-  acessar a lista de produtos
  acessandoListaProdutos(){
    this.cestaProdutos = this.objDoService.getProdutos()
  }
}
```

Primeiro, foram importadas a classes *Produto* & *ProductService* – a partir de seus arquivos de origem.

No construtor do *AppComponent,* foi criada instância da classe *ProductService().*

O método *acessandoListaProdutos()* chama o método *getProdutos()* implementado dentro do arquivo *product.service.ts*. A lista retornada de *Produtos* é armazenada na variável local *cestaProdutos*

**Criando a view**

O próximo passo é exibir *Produtos* para o usuário. Abra o arquivo ***app.component.html*** e adicione o seguinte código abaixo, como se segue:

***app.component.html***

```
<!--criar a tabela para exibir os dados -->
  <table class = "bordaTabela">
    <thead>
      <tr>
        <th>Identificação do Produto</th>
        <th>Nome do Produto</th>
        <th>Preço do Produto</th>
      </tr>
    </thead>
    <tbody>
      <tr *ngFor = "let p of cestaProdutos">
        <td>{{ p.idProduto }}</td>
        <td>{{ p.nomeProduto }}</td>
        <td>{{ p.precoProduto | currency: 'BRL' }}</td>
      </tr>
    </tbody>
  </table>
```

No código acima foi adicionado um botão chamado *"Obter Produtos"*, que é vinculado ao método *acessandoListaProdutos()* da classe do componente por meio de *event binding* (associação de eventos).

Os produtos são exibidos/retornados via diretiva ***ngFor***.

Por fim, execute o código e clique no botão "*Obter Produtos"* e observe o resultado no browser. Deve ser semelhante ao indicado abaixo:

Angular

Ver lista de Produtos

# Ola, angular-service

**Implementação services**

| Identificação do produto | Nome do produto | Preço do produto |
|---|---|---|
| 1 | Quadro Baby Yoda - Grogu | 199.9 |
| 2 | Mascara Darth Vader | 158.4 |
| 3 | Sabre de Luz | 89.99 |
| 4 | Action Figure Storm Trooper | 79.95 |
| 5 | Action Figure Princesa Leia | 199.99 |
| 6 | Action Figure Hans Solo | 199.99 |
| 7 | Miniatura Estrela da Morte | 59.98 |

Aplicação funfando! 🎉

**Injetando services no componente**

No projeto *Services* implementado acima, *productService* foi instanciado no componente diretamente, como mostrado abaixo:

```
constructor(){
    this.objDoService = new ProductService();
  }
```

A instanciação direta do service, como mostrado acima, tem muitas desvantagens

1. O *ProductService* está firmemente acoplado ao componente. Se a definição da classe *ProductService* for alterada, será necessário atualizar todos os códigos em que o service é usado
2. Se, por exemplo, a classe *ProductService* tiver seu nome alterado para para *NovoProductService,* será necessário procurar o nome “antigo” ProductService e, manualmente, alterá-lo
3. Acoplar a instaciação da classe ProductService diretamente no componenete faz com que os testes sejam mais difíceis para configuração e execução.

Esse problema pode ser resolvido por injeção de dependência.

## Angular Dependency Injection

A Angular Dependency Injection agora é parte central do Angular e permite que as dependências sejam injetadas no componente ou classe. Neste tutorial, aprenderemos o que é Angular Dependency Injection e como injetar dependência em um componente ou classe usando um exemplo

**O que é dependência**

O service *ProductService* foi construido utilizando Angular Services. O *AppComponent* é dependente do *ProductService* para fornecer a lista de produtos que será exibidio em tela para o usuário.

Em suma, *AppComponent* tem uma dependência em *ProductService*.

**Angular Dependency Injection -definição**

Injeção de Dependência (DI) é uma técnica na qual é fornecida uma instância de um objeto para outro objeto, que depende dele. Essa técnica também é conhecida como "Inversão de controle" (IoC)

Observe o service – *ProductService* - criado anteriormente: *ProductService* retorna os produtos dispostos dentro do *array* quando o método *getProdutos* é chamado. Observe, novamente, a estrutura de código abaixo:

```
import { Produto } from './product'

export class ProductService {

 // criar uma função para retornar um lista de produtos
 public getProdutos(){

   // propriedade que será a lista de produtos
   let listaProdutos: Produto[]

   // criando os itens da lista de produtos
   listaProdutos = [
      new Produto(1, 'Quadro Baby Yoda', 199.89),

      new Produto(2, 'Mascara Darth Vader', 159.79),

      new Produto(3, 'LightSaber', 89.99),

      new Produto(4, 'Estrela da Morte Miniatura', 39.69),
      new Produto(5, 'Boneco Storm Trooper', 59.25),
      new Produto(6, 'Miniatura Princesa Leia', 59.99),
```

```
      new Produto(7, 'Miniatura Han Solo', 19.89)
    ]

    // retorno da lista montada/criada
    return listaProdutos
  }
}
```

***productService*** foi instaciado diretamente dentro do arquivo *app.component.ts*. Observe, novamente, o código abaixo:

```
import { Component } from '@angular/core';
// importar os recursos necessarios para consumir o service
import { ProductService } from '../service/product.service';
import { Produto } from '../model/produto';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']

})
export class AppComponent{

  tituloComp: string = 'Implementação Services'

  // casting das propriedades que serão usadas
  cestaProdutos!:Produto[]
  objDoService


  //chamando o construtor
 constructor(){
     this.objDoService = new ProductService();
   }

  // criando uma função para - através do uso do objeto criado
-  acessar a lista de produtos
  acessandoListaProdutos(){
    this.cestaProdutos = this.objDoService.getProdutos()
  }

}
```

A instância *ProductService* possui o escopo de “instância local” para o componente. *CServiceComponent,* agora, está fortemente acoplado ao ***ProductService.*** Este acoplamento rígido pode trazer problemas – como visto no texto acima.

Idealmente, é necessário criar um service ***ProductServiceSingleton*** para que possa ser usado na aplicação.

Para minimizar os problemas enfrentados com o escopo local do *service* criado basta mover a instanciação de ***ProductService*** para o construtor da classe ***CServiceComponent*****,** como mostrado abaixo:

```
  //chamando o construtor
 constructor(private objDoService: ProductService){

 }
```

Agora, *CServiceComponent* não cria a instância do *ProductService*. Ele apenas “pede” em seu Construtor. *AppComponent* passa a ser dissociado do *ProductService*; “não sabe nada” sobre o *ProductService*. Apenas funciona com o ProductService passado para ele. Dessa forma, é possível passar qualquer alteração que seja feita em qualquer service, por exemplo: *ProductService*, *BetterProductService*, *MockProductService*.

O padrão acima é conhecido como ***Dependency Injection Pattern (injection pattern de dependência).***

**Importância de se usar injeção de dependência**

O componente agora está fracamente acoplado ao *ProductService*. *AppComponent* “não sabe” como criar o *ProductService****.***

*AppComponent* não depende mais de uma implementação particular *ProductService*. Funcionará com qualquer implementação *ProductService* transmitida a ele. Para, por exemplo, criar teste unitários basta, apenas, criar uma classe *mockProductService* e passá-la durante o teste.

A reutilização do componente fica mais fácil. O componente agora funcionará com qualquer ***ProductService***, desde que a interface seja respeitada.

O denpendency injection pattern tornou nosso ***AppComponent*** “testável” (passível de testes), e passível de manutenção, etc.

Ainda, nesse momento, o problema todo não foi resolvido -somente minimizado. O problema foi movido do *Component* para o Criador do *Component.*

Se o projeto for salvo e executado com a alteração feita acima, nada será exibido em tela.

Como criamos uma instância ***ProductService*** e a passamos para o ***AppComponent***? É isso que ***Angular Dependency Injection*** faz.

**Estrutura da Angular Dependency Injection**

A estrutura de Angular Dependency Injection implementa o Injection pattern de Dependência em Angular. Ele cria e mantém as Dependências e as “injeta” nos Components ou Services que solicitam.

**Partes da estrutura de Angular Dependency Injection**

Existem cinco partes principais da estrutura de Angular Dependency Injection.

**Consumer (Consumidor)**

É o componente que precisa da dependência. No exemplo acima, o AppComponent é o Consumidor

**Dependency (Dependência)**

O serviço que está sendo “injetado”. No exemplo acima, *ProductService* é a Dependency

**DI Token**

O DI Token identifica exclusivamente uma Dependência. Usamos o DI Token quando registramos dependência

**Provider (Fornecedor)**

Os providers (fornecedores) mantém a lista de dependências junto com seus tokens. O DI Token é usado para identificar a Dependência.

**Injector**

O *Injector* é o mantenedor dos *Providers* e é responsável por resolver as dependências e injetar a instância da Dependency (dependência) para o Consumer (Consumidor)

**Como funciona a injeção de dependência no Angular**
**conceito de Inject, Injector e Injectable**

As dependências são registradas com o *Provider*. Isso é feito nos *Providers* metadados do *Injector*.

O Angular fornece uma instância de *Injector* & *Provider* para todo *consumer* (consumidor).

O *consumer* (consumidor), quando instanciado, declara as dependências de que precisa em seu construtor.

O *Injector* lê as dependências do construtor do *consumer* (consumidor) e procura a dependência no *provider* (provedor). O provider (provedor) fornece a instância e o injector e injeta no consumer (consumidor). Se a instância da Dependency já existir, ela será reutilizada, tornando a dependência única.

**Como usar a dependency Injection (injeção de dependência)**

Foi criado um ***ProductService*** no último passo-a-passo. Vamos atualizá-lo, agora, para usar a dependency injection (Injeção de Dependência).

Primeiro, precisamos registrar as dependências no provider. Isso é feito no array de metadados *providers* do *decorator* @Component.

```
providers: [ProductService]
```

Observe o código abaixo. Implemente-o como se segue:

***app.component.ts***

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { ProductService } from './services/produto.service';

import { Produto } from './models/produto';
import { CommonModule } from '@angular/common';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet, CommonModule],
  templateUrl: './app.component.html',
  styleUrl: './app.component.css',
  providers:[ProductService]
})
export class AppComponent {
  title = 'angular-service';

  tituloComp: string = 'Implementação services'

  cestaProdutos!: Produto[]
  constructor(private objService: ProdutoService){

  }

  acessarListaDados(): void{
    this.cestaProdutos = this.objService.listagemProdutos()
  }
}
```

Assim, com o registro do service dentro do array providers, é possível referencia a *dependency injection* no construtor de *AppComponent*. Salve o projeto e execute. Veja o resultado no browser. Deve ser semelhante ao indicado abaixo:

Angular

Ver lista de Produtos

# Ola, angular-service

**Implementação services**

| Identificação do produto | Nome do produto | Preço do produto |
|---|---|---|
| 1 | Quadro Baby Yoda - Grogu | 199.9 |
| 2 | Mascara Darth Vader | 158.4 |
| 3 | Sabre de Luz | 89.99 |
| 4 | Action Figure Storm Trooper | 79.95 |
| 5 | Action Figure Princesa Leia | 199.99 |
| 6 | Action Figure Hans Solo | 199.99 |
| 7 | Miniatura Estrela da Morte | 59.98 |

Aplicação funfando! 🎉

Em seguida, precisamos informar angular que nosso componente precisa de “injeção” de dependência. Isso é feito usando o decorator ***@Injectable().***

O *decorator* ***@Injectable()*** não é necessário, se a classe já tem outros *decorators.* O Angular utiliza: ***@Component***, ***@pipe*** ou ***@directive***, entre outros. Porque todos estes são um subtipo de ***Injectable***.

Como ***AppComponent*** já está “decorado” com ***@Component***, não é necessário “decorar” com o ***@Injectable***

*Nota: @Injectable também não é necessário se a classe não tiver nenhuma dependência a ser injetada. No entanto, a melhor prática é “decorar” todas as classes de service @Injectable(), mesmo aquelas que não têm dependências .*

**Injetando Service em Service**

Até este momento, foi observado como injetar *ProductService* a um componente. Agora, observe como injetar um service em outro service.

Crei um novo *service* e nomeie-o como *testando*, que registra todas as operações em uma janela do console e as injeta em *ProductService*. Para criar o novo service, abra o prompt de comando – via *vs code* ou pelo próprio

computador – e, dentro da pasta de seu projeto (*angular-alpha*), insira a seguinte instrução:

```
ng g service nomedoservice
ng g service testando
```

```
angular-alpha>ng g service testando
CREATE src/app/testando.service.spec.ts (357 bytes)
CREATE src/app/testando.service.ts (135 bytes)
```

Agoram abra o novo arquivo service criado – *testando.service.ts* – e implemente o código abaixo como se segue:

***testando.service.ts***

```
import { Injectable } from '@angular/core';

@Injectable()
export class TestandoService {
  // função para verificar se ProductService está sendo
executado corretamente
  unitario(mensagemTeste:any){
    console.log(mensagemTeste)
  }
}
```

A classe ***TestandoService*** possui apenas um método nomeado como *log()*, que captura o valor atribuído ao argumento *mensagemTeste* e exibe no console.

Também está sendo usado o decorator, com os metadados, *@Injectable* para “decorar” a classe *TestandoService*. Tecnicamente, isso não é necessário aqui, pois a classe não possui nenhuma dependência externa. É uma boa prática adicionar metadados @Injectable pelos seguintes motivos:

- provas futuras: Não é necessário lembrar @Injectable() quando adicionamos uma dependência posteriormente.
- Consistência: todos os services seguem as mesmas regras e não precisamos nos perguntar por que um decorator está ausente.

**Inject na classe ProductService**

Agora, o próximo passo é injetar a classe ***TestandoService*** na classe ***ProductService***

***ProductService*** necessita que *TestandoService* seja injetado. Portanto, a classe requer metadados *@Injectable*. Observe a instrução abaixo e implemente-a como se segue – faça a importação, também, de *TestandoService* e *Injectable*:

```
import { Produto } from './product'
import { TestandoService } from './testando.service';
import { Injectable } from '@angular/core';

@Injectable()
```

Na sequência, no construtor de ***ProductService*** é feita a solicitação de ***TestandoService***. Crie o construtor da classe e implemente o código abaixo como se segue:

```
    // implementando o construtor
 constructor(private resultTeste: TestandoService){
    this.resultTeste.unitario('Construtor
chamado/construido com sucesso!')
   }
```

Altere o método ***GetProdutos*** para usar o objeto ***resultTeste***.

```
/ criar uma função para retornar um lista de produtos
 public getProdutos(){

    this.resultTeste.unitario('Método/função getProdutos()
chamado superultra top!')

   // propriedade que será a lista de produtos
   let listaProdutos: Produto[]

   // criando os itens da lista de produtos
   listaProdutos = [
     new Produto(1, 'Quadro Baby Yoda', 199),
     new Produto(2, 'Mascara Darth Vader', 159),
     new Produto(3, 'LightSaber', 89)
   ]
```

```
    this.resultTeste.unitario(listaProdutos)

    // retorno da lista montada/criada
    return listaProdutos
  }
```

Finalmente, será necessário registrar ***TestandoService*** com os metadados em ***Providers***. Abra o *app.module.ts* para atualizar o providers e incula o registro de ***TestandoService.*** Observe o código abaixo e implemente-o como indicado:

***app.component.ts***

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { ProductService } from './services/produto.service';
import { TestandoService } from './services/testando.service';


import { Produto } from './models/produto';
import { CommonModule } from '@angular/common';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet, CommonModule],
  templateUrl: './app.component.html',
  styleUrl: './app.component.css',
  providers:[ProductService, TesttandoService]
})
export class AppComponent {
  title = 'angular-service';

  tituloComp: string = 'Implementação services'

  cestaProdutos!: Produto[]
  constructor(private objService: ProdutoService){

  }

  acessarListaDados(): void{
    this.cestaProdutos = this.objService.listagemProdutos()
  }
}
```

Ao clicar no botão “Obter Produtos”, é possível observar pelo console do browser com as respectivas mensagens de log. Salve o projeto e observe o resultado. Deve ser semelhante ao indicado abaixo:

Angular

Ver lista de Produtos

# Ola, angular-service

**Implementação services**

| Identificação do produto | Nome do produto | Preço do produto |
|---|---|---|
| 1 | Quadro Baby Yoda - Grogu | 199.9 |
| 2 | Mascara Darth Vader | 158.4 |
| 3 | Sabre de Luz | 89.99 |
| 4 | Action Figure Storm Trooper | 79.95 |
| 5 | Action Figure Princesa Leia | 199.99 |
| 6 | Action Figure Hans Solo | 199.99 |
| 7 | Miniatura Estrela da Morte | 59.98 |

Aplicação funfando! 🎉

```
Construtor chamado/construido com sucesso
Angular is running in development mode.
Método/função chamada/construida com sucesso
▼ Array(7) ℹ
  ▶ 0: Produto {idProduto: 1, nomeProduto: 'Quadro Baby Yoda - Grogu', precoProduto: 199.9}
  ▶ 1: Produto {idProduto: 2, nomeProduto: 'Mascara Darth Vader', precoProduto: 158.4}
  ▶ 2: Produto {idProduto: 3, nomeProduto: 'Sabre de Luz', precoProduto: 89.99}
  ▶ 3: Produto {idProduto: 4, nomeProduto: 'Action Figure Storm Trooper', precoProduto: 79.95}
  ▶ 4: Produto {idProduto: 5, nomeProduto: 'Action Figure Princesa Leia', precoProduto: 199.99}
  ▶ 5: Produto {idProduto: 6, nomeProduto: 'Action Figure Hans Solo', precoProduto: 199.99}
  ▶ 6: Produto {idProduto: 7, nomeProduto: 'Miniatura Estrela da Morte', precoProduto: 59.98}
    length: 7
  ▶ [[Prototype]]: Array(0)
```

O fornecimento do *service* no módulo raiz cria uma instância única e compartilhada desse *service* e injetará em qualquer classe que solicitar.

O código acima funciona porque, o Angular, cria a Árvore dos Injetores com o relacionamento pai-filho semelhante à Árvore dos Componentes.

***Obs.: Os serviços injetados no nível do módulo têm escopo no aplicativo, o*** *que significa que eles podem ser acessados de todos os componentes / serviços do aplicativo. Qualquer serviço fornecido no* ***Módulo filho*** *está disponível em todo o aplicativo.*

***Os serviços são fornecidos em um módulo lento*** *, com escopo definido no módulo e disponíveis apenas para o módulo carregado lento.*

***Os serviços fornecidos no nível do componente*** *estão disponíveis apenas para o componente e para os componentes filho.*

Onde você registra sua dependência, define o escopo da dependência. A dependência registrada no Módulo usando o decorator @NgModule é anexada ao Provedor Root (Provedor anexado ao InjectorRoot). Essa dependência está disponível para todo o aplicativo.

A dependência registrada com o componente está disponível para esse componente e para qualquer componente filho desse componente.

***ProductService*** possue uma relação de dependência do ***LoggerService***. Por isso, é "decorado" com o decorator @Injectable. Remova *@Injectable()* de *ProductService* será indicado a seguinte exceção:

Uncaught Error: Can't resolve all parameters for ProductService: (?)

Isso porque, sem DI Angular não é possível saber como injetar ***TestandoService*** em ***ProductService*****.**

Remover *@Injectable()* de *TestandoService* não resultará em nenhum erro, pois não há dependência.

Os *components* e *directives* já estão "decorados" com @Component & @Directive. Esses decorators também dizem ao Angular para usar o DI, portanto você não precisa adicionar o @Injectable().

## Referencias:

*adaptado do original que pode ser obtido pelo link abaixo*
*Alexandre Beato: https://medium.com/@alexandrebeato/pt-br-arquitetura-para-projeto-em-angular-com-f%C3%A1cil-desenvolvimento-e-manuten%C3%A7%C3%A3o-817a15e0d10c*

*https://blog.bitsrc.io/data-binding-in-angular-cbc433481cec*

*https://www.c-sharpcorner.com/article/learn-angular-8-step-by-step-in-10-days-data-binding-day-3/*

*Traduzido e adaptado do original que pode ser obtido através do link abaixo*
*https://www.tektutorialshub.com/angular/angular-directives/*

*Traduzido e adaptado do original que pode ser obtido em:*
*https://www.tektutorialshub.com/angular/angular-services/*
*Traduzido e adaptado do original que pode ser obtido pelo link abaixo*
*https://www.tektutorialshub.com/angular/angular-dependency-injection/*
*Traduzido e adaptado do original que pode ser obtido pelo link abaixo:*
*https://www.tektutorialshub.com/angular/angular-injector-injectable-inject/*
*Traduzido e adaptado do original que pode ser obtido pelo link abaixo:*
*https://www.tekpasso-a-passoshub.com/angular/angular-providers/*
*Traduzido e adaptado do original que pode ser obtido pelo link abaixo:*
*https://www.tekpasso-a-passoshub.com/angular/angular-hierarchical-dependency-injection/*
*Traduzido e adaptado do original que pode ser obtido através do link abaixo:*
*https://medium.com/@jinalshah999/reactjs-step-by-step-tutorial-series-for-absolute-beginners-part-1-9f727d72d93*
*https://medium.com/@jinalshah999/reactjs-step-by-step-tutorial-series-part-2-reactjs-components-7b6a5078f824*
*https://medium.com/@jinalshah999/reactjs-step-by-step-tutorial-series-part-3-reactjs-components-communication-2c0707e90ad4*